Сортировки за $O(n^2)$

{ Пузырьком;
Вставками;
Выбором;
}

Давайте рассмотрим сортировку выбором

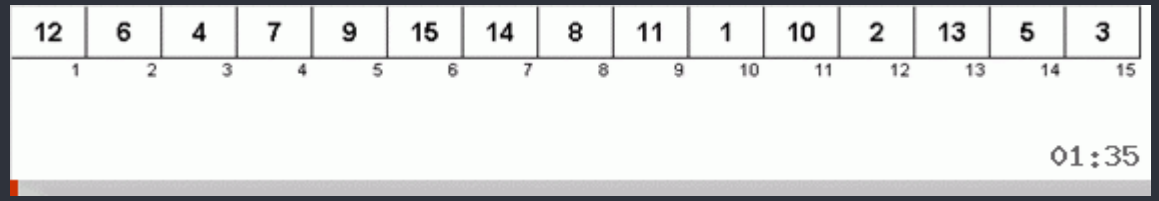
01 {

[Сортировка выбором]

< Сортировка выбором находит максимальный элемент и перемещает его в конец массива. >

}

Визуализация < /1 > {



}

Пояснение < /2 > {

< Находим максимум в оставшейся части массива и меняем его местами с последним элементом.

Сложность: $O(n^2)$, так как мы выполняем n итераций поиска максимума. >

}

Посмотрим псевдокод

SelectionSort; {

```
SelectionSort(array):
    n = длина(array)
    для i от 0 до n - 1:
        minIndex = i
        для j от i + 1 до n:
            если array[j] < array[minIndex]:
                minIndex = j
        обменять array[i] и array[minIndex]

    < Сложность: O(n²), так как мы выполняем n итераций
    внешнего цикла и до nn итераций внутреннего. >
```

}

Пирамида.html РПЛ.css

```
Рассмотрим чемпионат
'РПЛ' {
    [Каждая команда играет с
    каждой дважды - на своем поле
    и на поле соперника]
        < По сумме набранных очков выбирается
        победитель - сложность O(n²) >
}
```

Давайте рассмотрим кубковую систему с игрой навылет на примере Евро 2024

Пирамида.html

Евро 2024.css

1
2
3
4
5
6
7
8
9
10
11
12
13
14

Можем ли мы определить самую сильную сборную Европы в 2024 году?

Примечание { Для нашего удобства будем считать, что кубковая система обладает свойством транзитивности:

=> если Германия выиграла Данию
=> Испания выиграла Германию
=> Испания сильнее Дании.

}

Пирамида.html

Евро 2024.css

1
2
3
4
5
6
7
8
9
10
11
12
13
14

Какое количество игр потребовалось сыграть для выявления сильнейшей сборной?

Пирамида.html

Евро 2024.css

1
2
3
4
5
6
7
8
9
10
11
12
13
14

$O(n*\log_2 n)$?

Пирамида.html

Евро 2024.css

1
2
3
4
5
6
7
8
9
10
11
12
13
14

Можем ли мы определить вторую по силе сборную?

Пирамида.html

Евро 2024.css

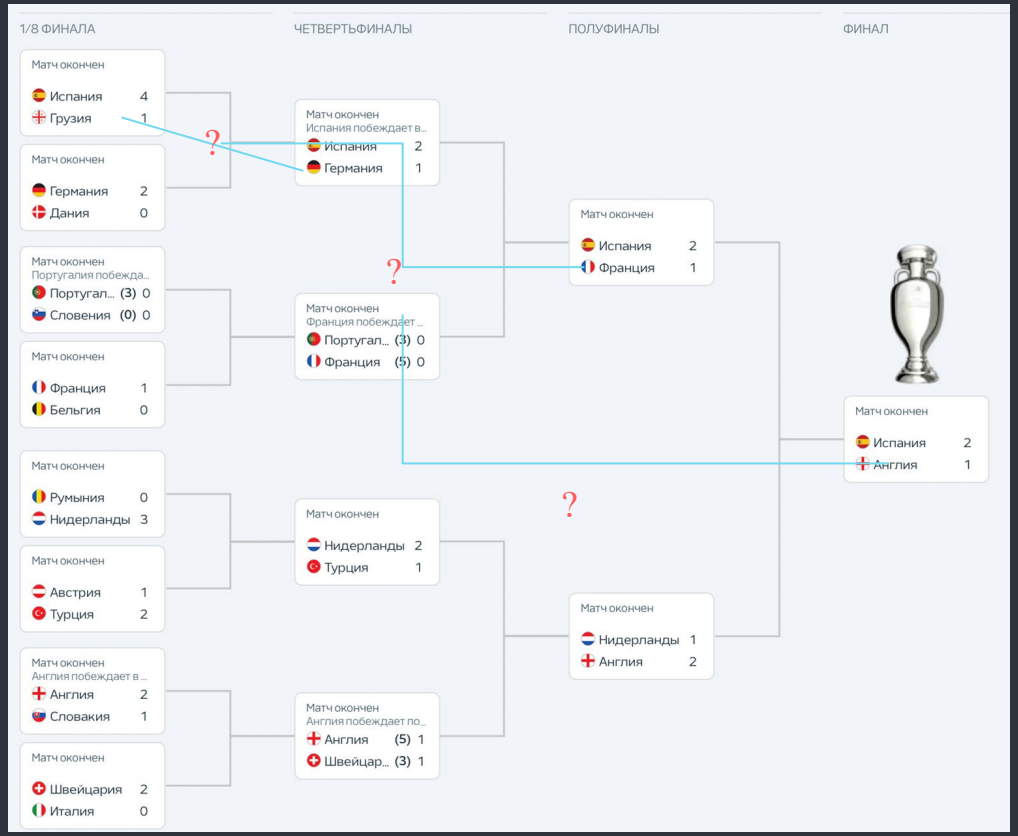
1
2
3
4
5
6
7
8
9
10
11
12
13
14

Что для этого требуется?

Пирамида.html

Евро 2024.css

1
2
3
4
5
6
7
8
9
10
11
12
13
14

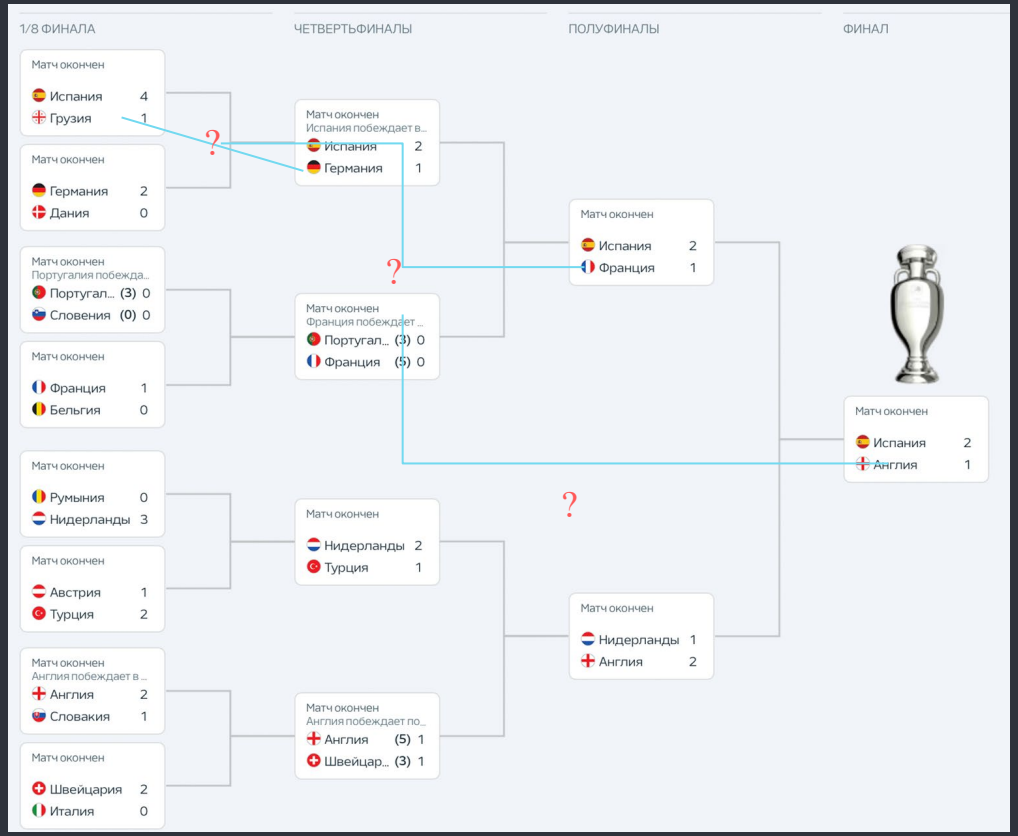


Быстрее чем O(n) ?

Пирамида.html

Евро 2024.css

1
2
3
4
5
6
7
8
9
10
11
12
13
14



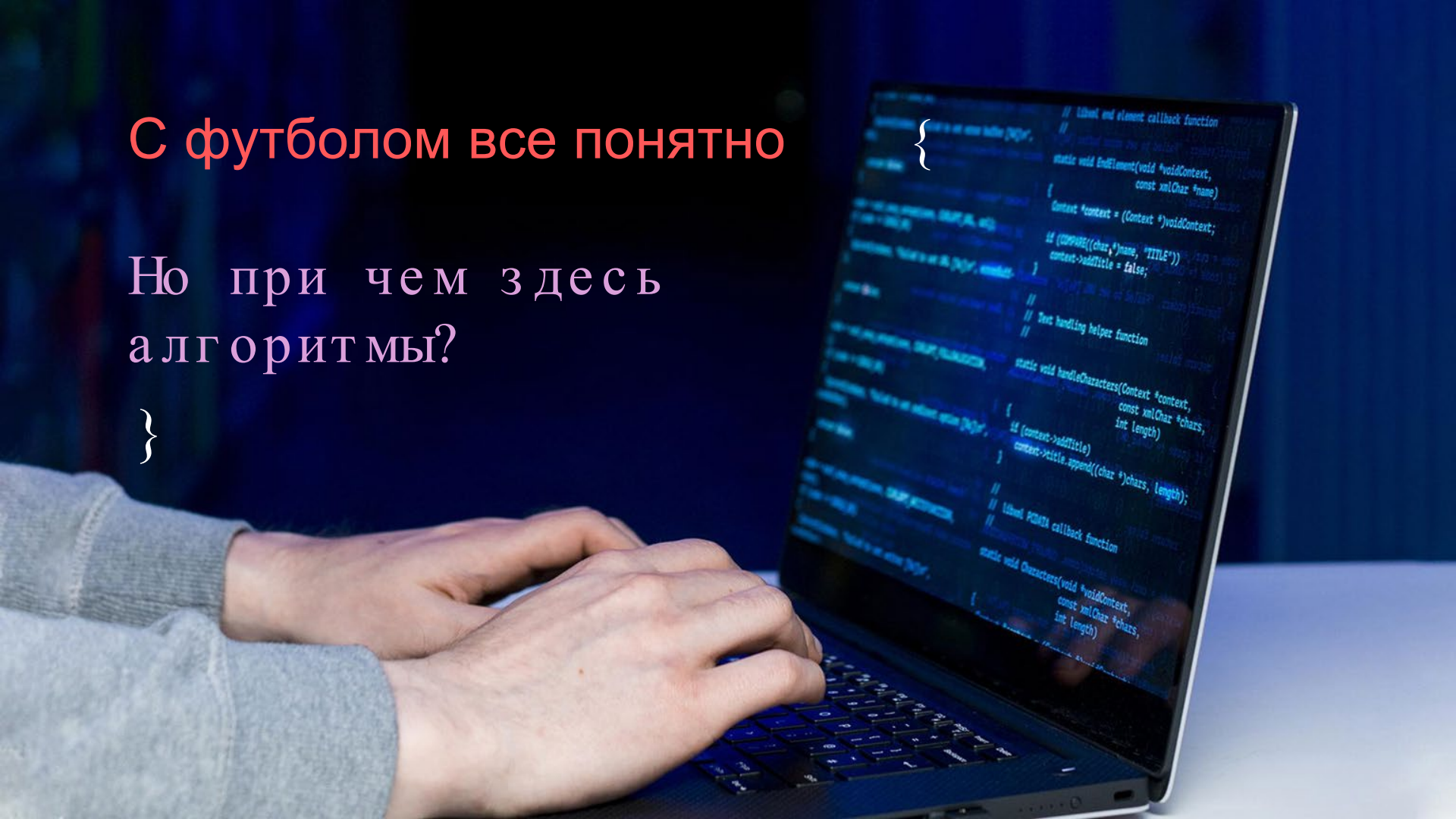
Предположительно все последующие места мы сможем определять за $O(\log_2 n)$

С футболом все понятно

{

Но при чем здесь алгоритмы?

}



Пирамида.html

Сортировка выбором.css

Вспомним сортировку выбором < /1 > {

< Мы n раз выполняли операцию поиска максимума в оставшейся части массива за O(n) >

}

А что если? < /2 > {

< А что если была бы такая структура данных, которая позволяла извлекать максимум за $O(\log_2 n)$ >

}

Так, допустим, мы хотим перейти от сравнения каждого элемента с каждым *(чемпионат)* к выбору максимума за $O(\log_2 n)$ *(кубок)* - при чем здесь пирамида?

Пирамида.html

Определение.css

Что такое ‘Пирамида / куча / Heap’ {

01 Завершенное бинарное дерево

< бинарное дерево, в котором каждый уровень, кроме последнего, полностью заполнен, а заполнение последнего уровня производится слева направо >

02 Если maxHeap

< Значение в любой вершине не меньше, чем значения её потомков >

03 Если minHeap

< Значение в любой вершине не больше, чем значения её потомков >

}

Много текста. Покажите картинку.

Завершенное бинарное дерево {

< В некоторых источниках такое дерево называют полным. Но будьте внимательны, полным бинарным деревом также называют бинарное дерево, в котором у каждого узла ноль или два дочерних узла >

}

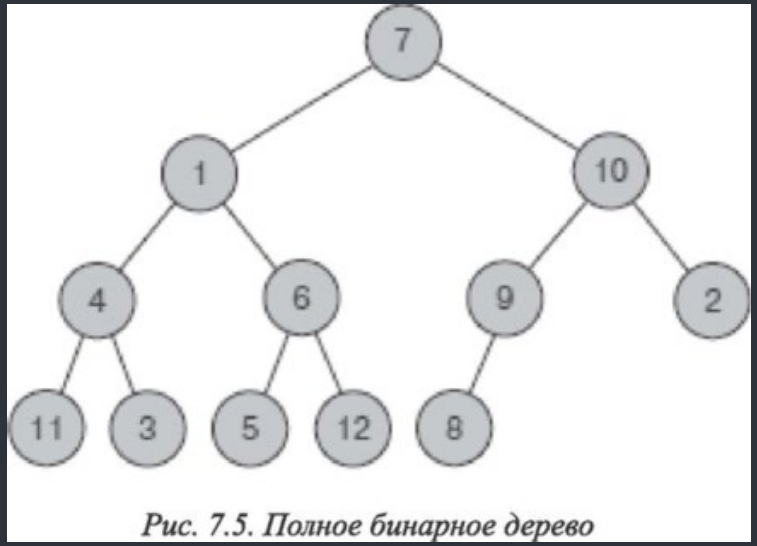
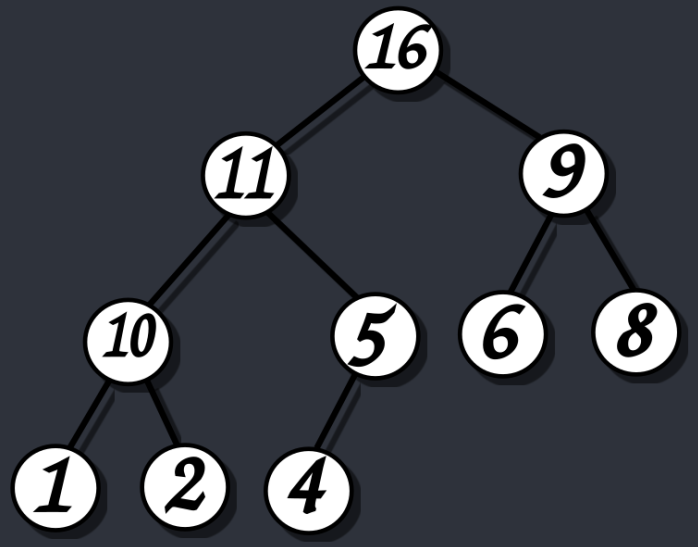


Рис. 7.5. Полное бинарное дерево

```
1
2
3
4  Рассмотрим
5  maxHeap {
6
7
8
9      < Завершенное бинарное дерево, в
10     котором значение в каждой вершине
11     больше или равно значению в его
12     потомках >
13
14 }
```



Как быстро найти максимум в такой структуре данных?

Пирамида.html

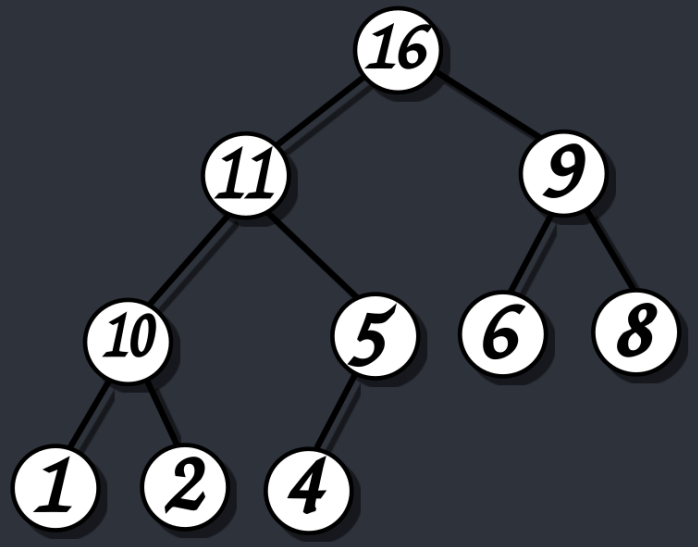
maxHeap.css

Рассмотрим maxHeap {

< Завершенное бинарное дерево, в котором значение в каждой вершине больше или равно значению в его потомках >

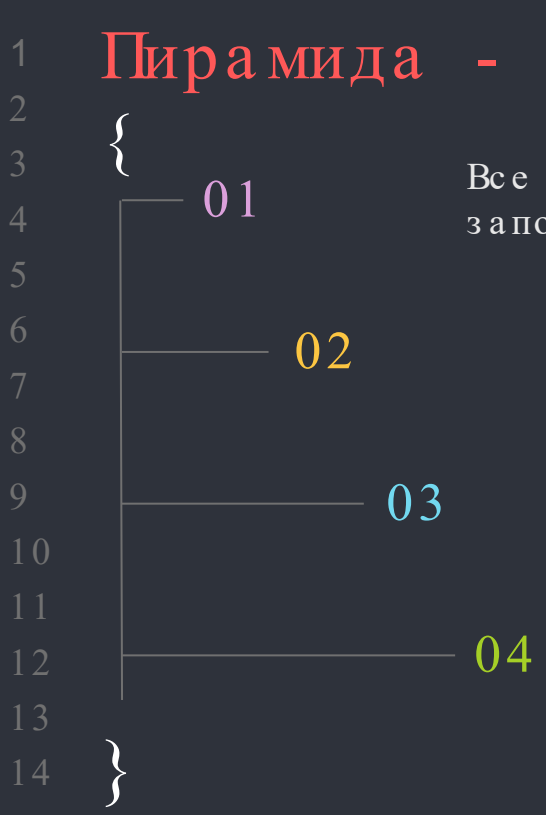
}

Максимум хранится в корне



Снова Node со значением и указателями на детей?

Пирамида - завершенное бинарное дерево



Все уровни заполнены, кроме последнего, который заполняется слева направо.

Массив идеально подходит для хранения такого дерева, так как его элементы можно размещать по индексам, не оставляя "пустых мест".

Формулы для узла с индексом i:

- Левый и правый потомки: $2*i+1$ и $2*i+2$
- Родитель: $\lfloor (i-1)/2 \rfloor$

Эти формулы работают за O(1) времени, что упрощает доступ к элементам пирамиды.

Какие ещё преимущества?

Пирамида.html

Преимущества массива.css

* Экономия памяти
  - Если использовать дерево (на указателях), каждому узлу потребуется хранить указатели на потомков и родителя.
    - Для n узлов потребуется O(n) дополнительной памяти на указатели.
  - Массив, напротив, не требует дополнительной памяти, так как индексы автоматически "связывают" узлы.
* Лёгкость реализации
  - Благодаря фиксированным формулам, операции на пирамиде (вставка, удаление, восстановление структуры) проще реализовать на массиве, чем на дереве.
* Эффективное использование памяти кэша
  - Массивы в памяти расположены последовательно, что позволяет эффективно использовать кэш процессора. Это ускоряет операции на пирамиде, так как обращение к соседним элементам требует меньше времени.

Не хочу читать. Покажите картинку.

Пирамида.html

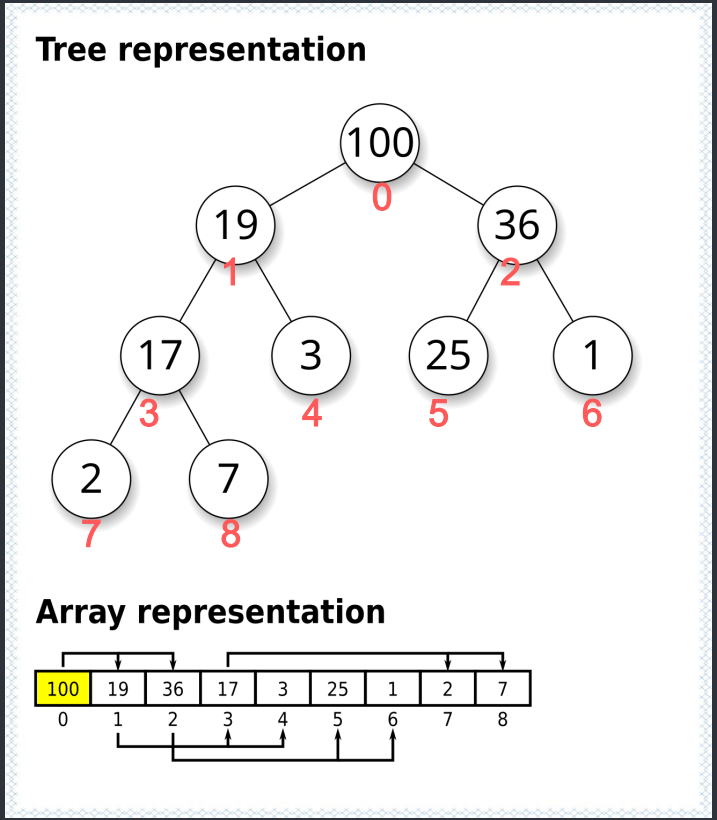
Представление в массиве.css

Пирамида на массиве {

Формулы для узла с индексом i:

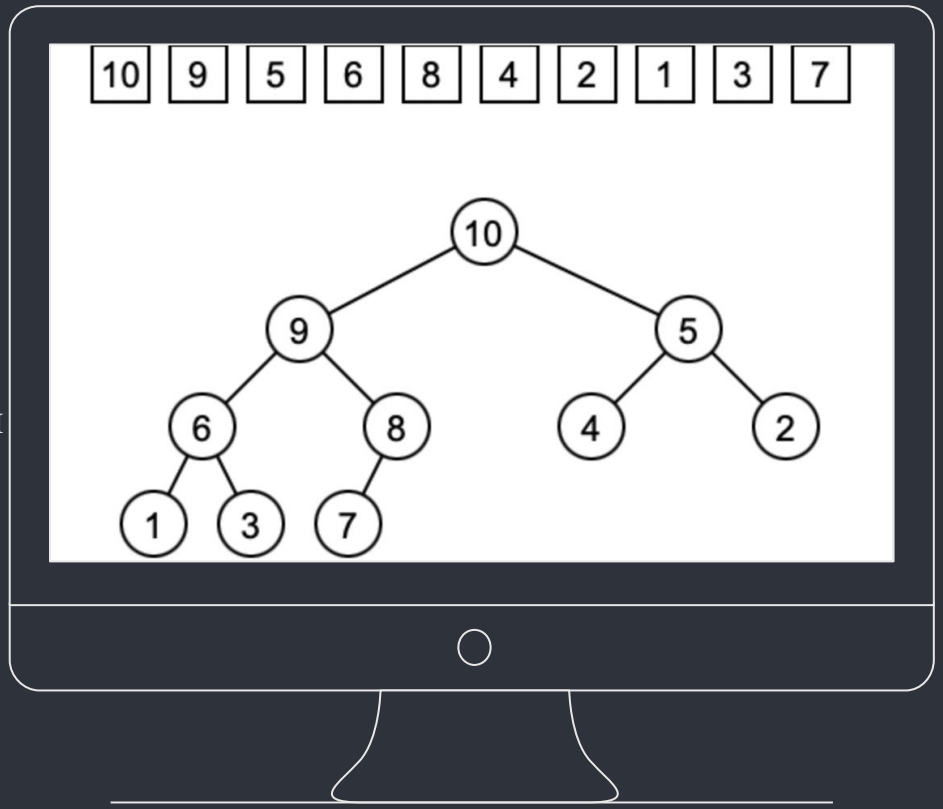
- Левый и правый потомки: $2*i+1$ и $2*i+2$
- Родитель: $\lfloor (i-1)/2 \rfloor$

}



А что там с сортировкой?

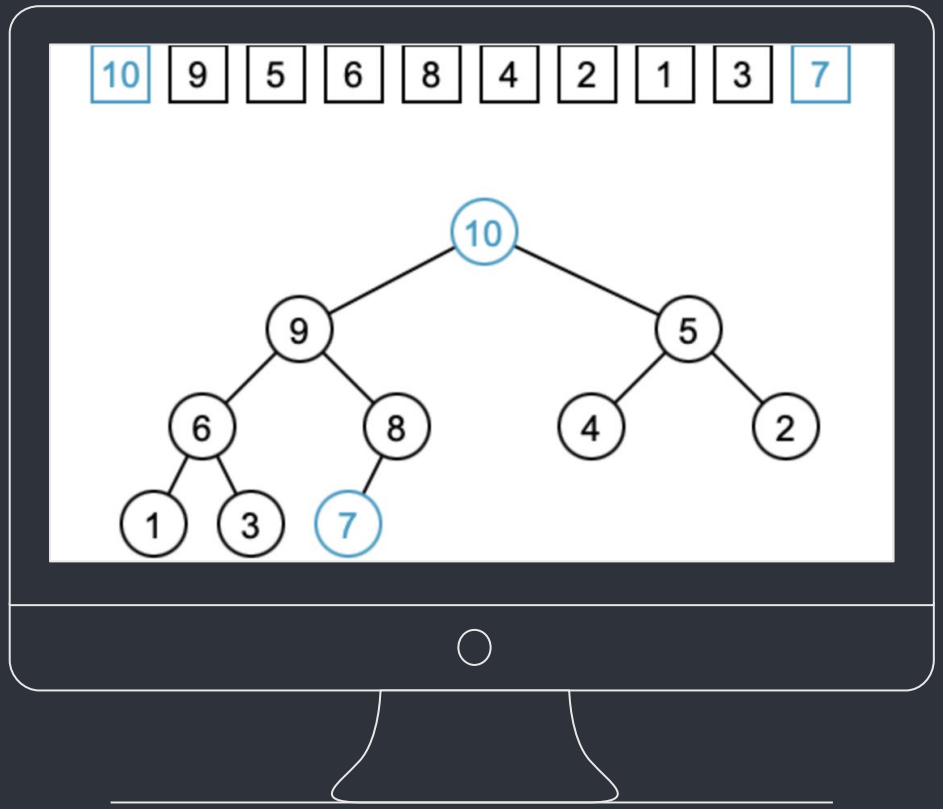
```
1
2
3
4  Max Heap {
5
6
7
8      Элемент с максимальным
9      значением лежит в
10     корне дерева => в
11     массиве в нулевом
12     индексе a[0]
13
14 }
```



Максимальный элемент в отсортированном массиве должен находиться в самом конце

```
1
2
3
4  MaxHeap {
5
6
7
8
9
10
11
12
13
14 }
```

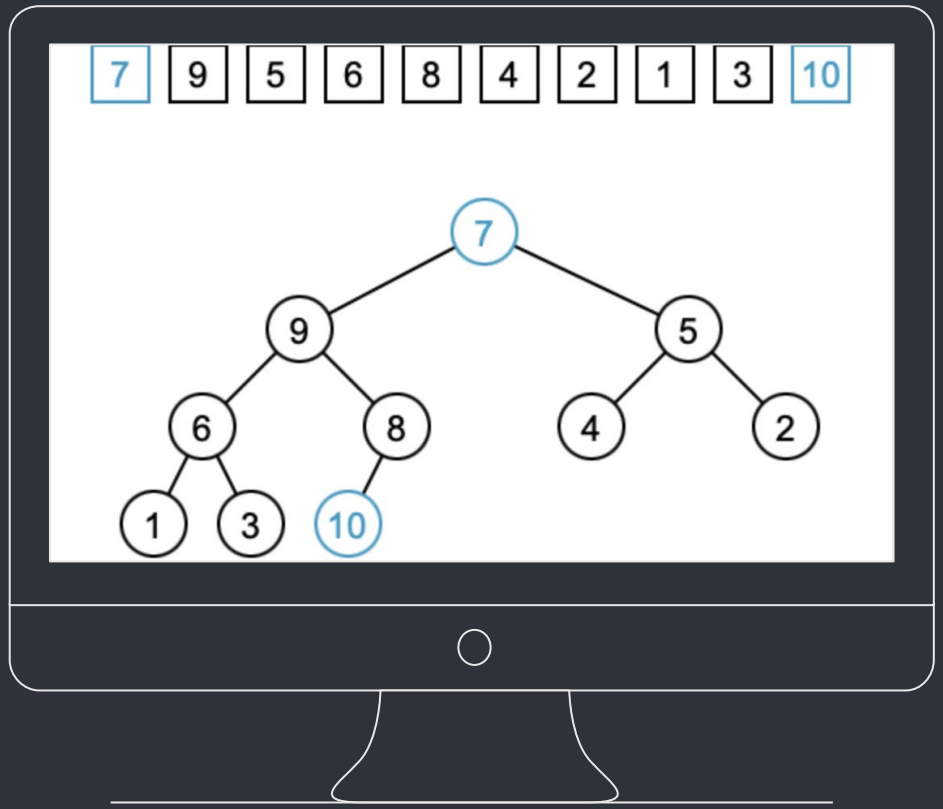
Поменяем первый и последний элемент местами



Swap(a[0], a[heapSize]) - это мы умеем

```
1
2
3
4  Max Heap {
5
6
7
8
9
10
11
12
13
14 }
```

Поменяли первый и последний элемент местами

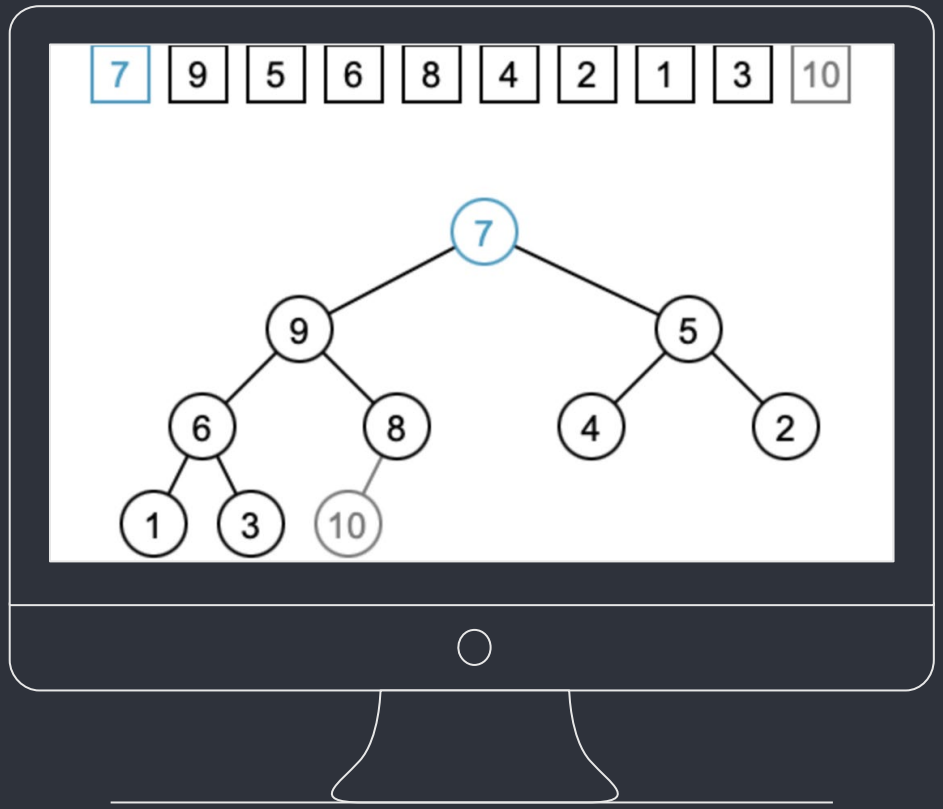


Последний элемент стоит на своем месте, давайте уберем его из кучи?

Пирамида.html

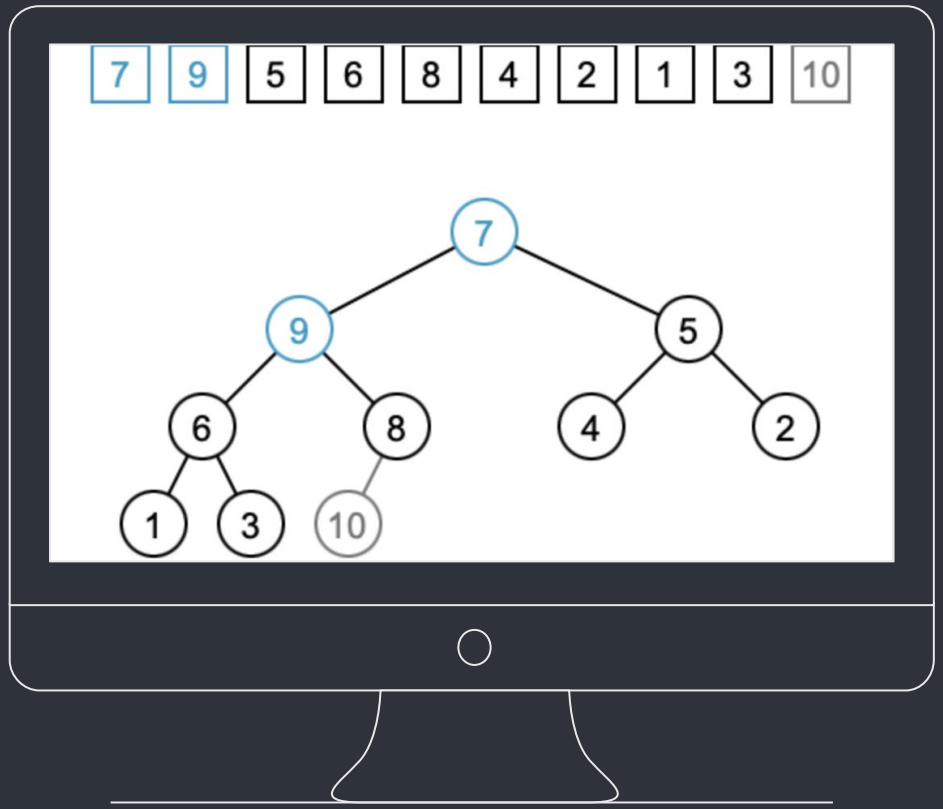
Пирамидальная сортировка.css

```
1
2
3
4  MaxHeap {
5
6
7
8
9
10     heapSize--
11
12
13
14 }
```



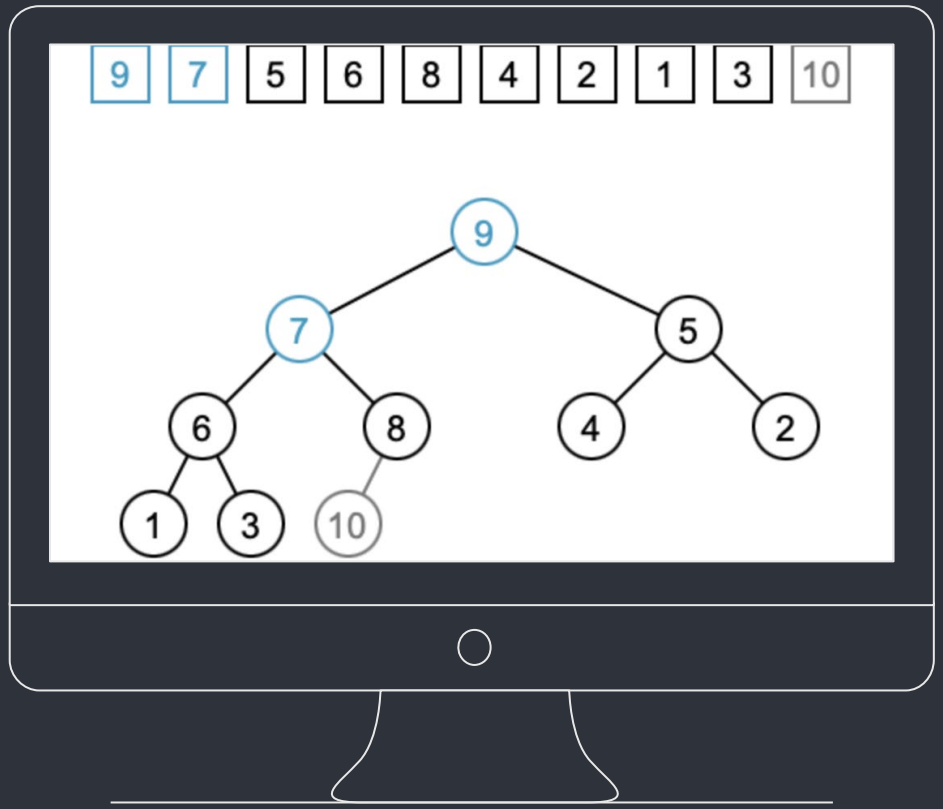
У корневого элемента левое и правое поддерево являются пирамидами - свойство кучи нарушается только в корне

```
Max Heap {
left = 2 * i + 1
right = 2 * i + 2
j = left

if (right < a.heapSize and
a[right] > a[left])
    j = right

if a[j] <= a[i]
    break

swap(a[i], a[j])

}
```



Операция siftDown - просеивание вниз

```
Max Heap {
left = 2 * i + 1
right = 2 * i + 2
j = left

if (right < a.heapSize and
a[right] > a[left])
    j = right

if a[j] <= a[i]
    break

swap(a[i], a[j])

}
```

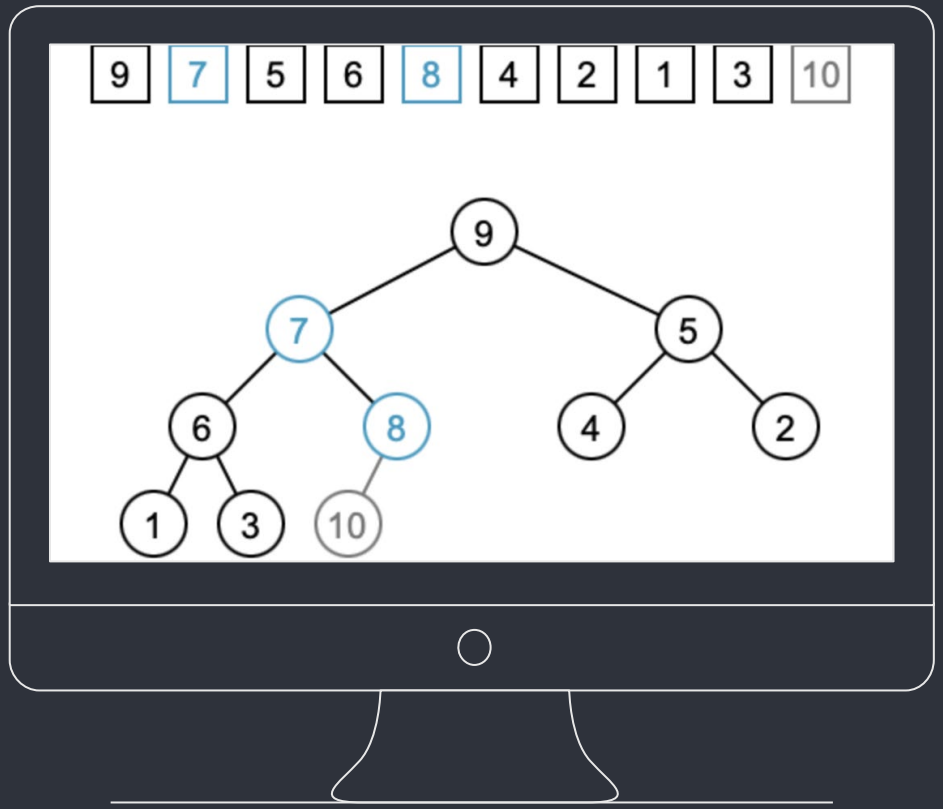


Сравниваем i элемент с большим из дочерних, если i элемент меньше дочернего - меняем их местами

Пирамида.html

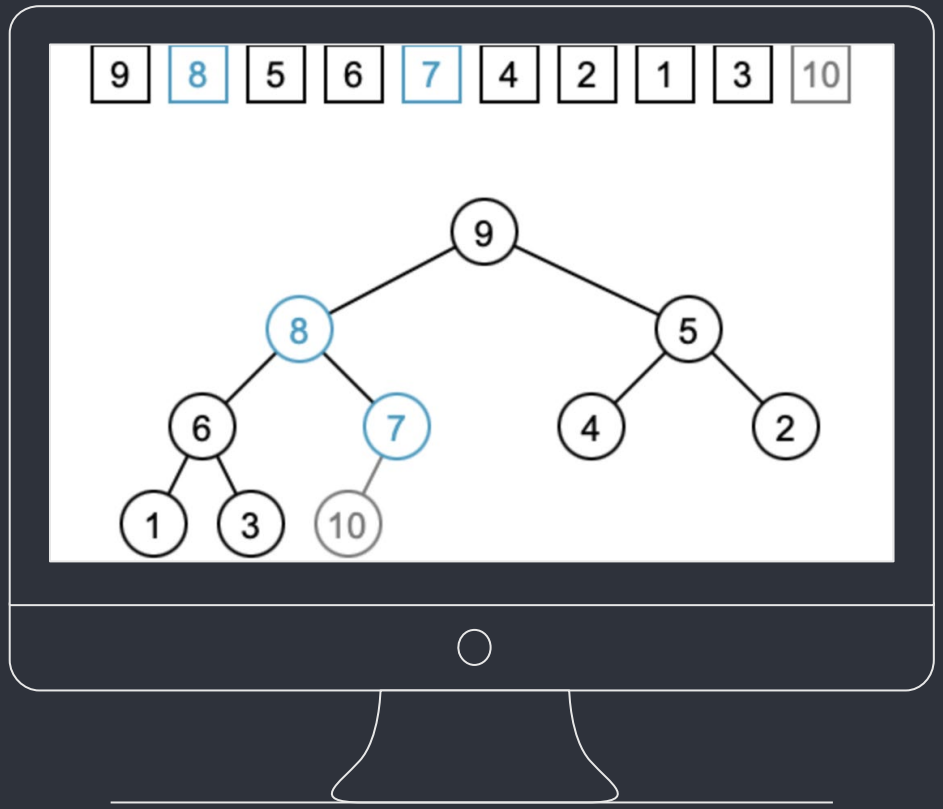
Пирамидальная сортировка.css

```
Max Heap {
left = 2 * i + 1
right = 2 * i + 2
j = left

if (right < a.heapSize and
a[right] > a[left])
    j = right

if a[j] <= a[i]
    break

swap(a[i], a[j])

}
```



Выполняем siftDown до тех пор, пока (i элемент не окажется >= дочерним) или (i элемент не станет листом)

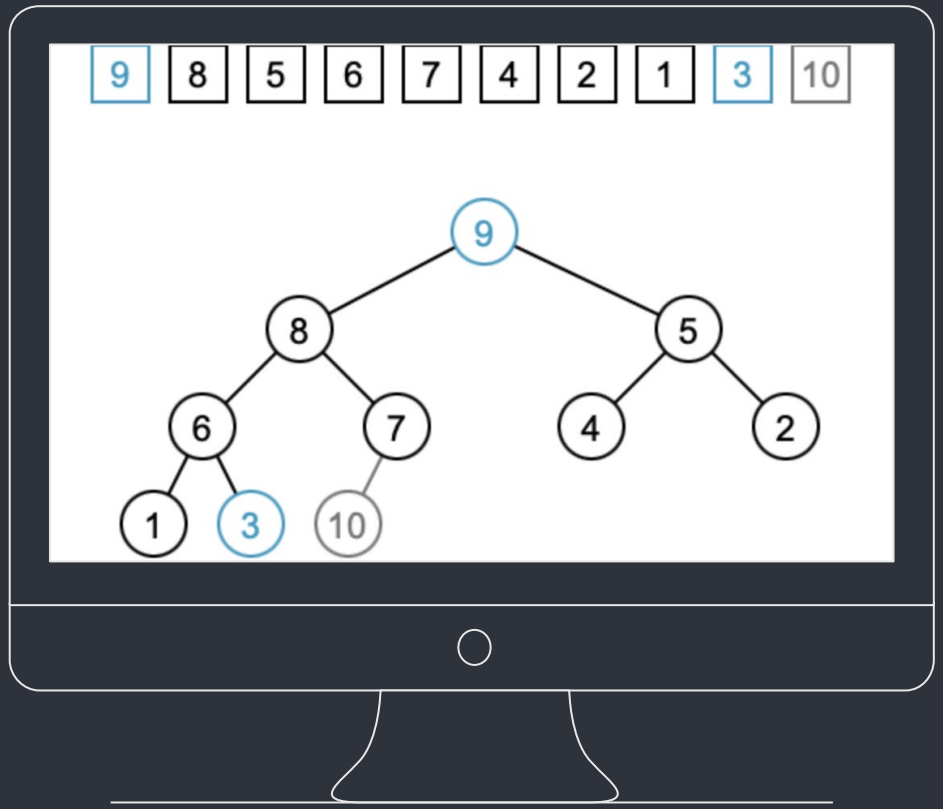
```
Max Heap {
left = 2 * i + 1
right = 2 * i + 2
j = left

if (right < a.heapSize and
a[right] > a[left])
    j = right

if a[j] <= a[i]
    break

swap(a[i], a[j])

}
```



Закончили упражнение - i элемент является листом

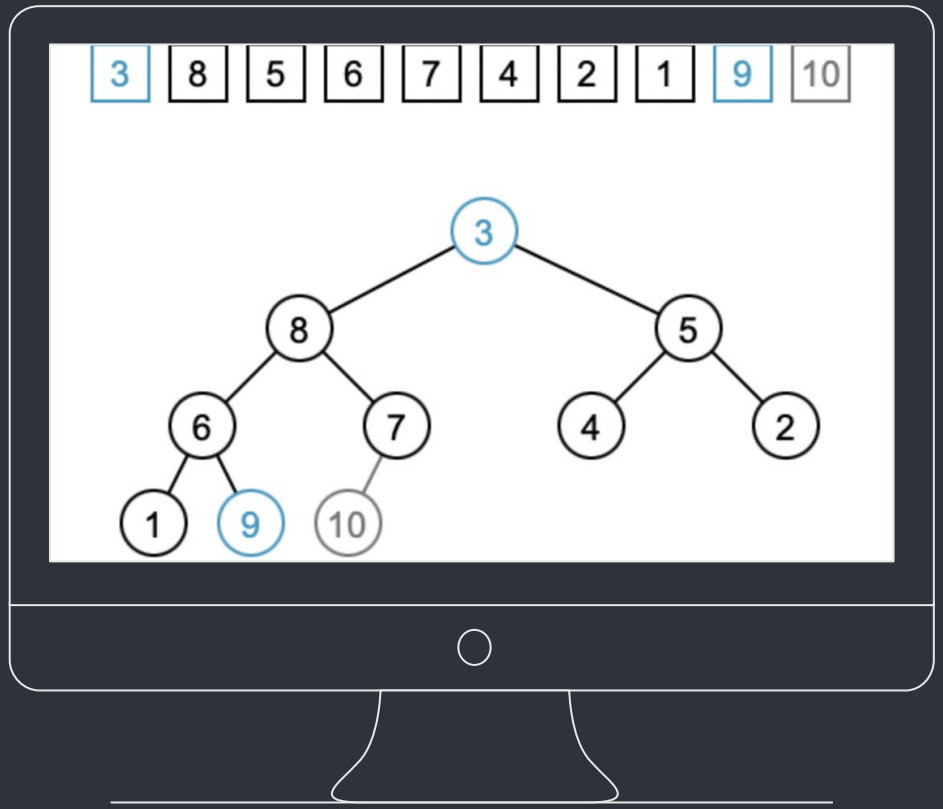
```
1
2
3
4  Max Heap {
5
6
7
8
9
10
11
12
13
14 }
```

Поменяем первый и последний элемент местами



Swap(a[0], a[heapSize]) - это мы умеем

```
1
2
3
4  Max Heap {
5
6
7
8
9      Поменяли первый и
10     последний элемент
11     местами
12
13
14 }
```

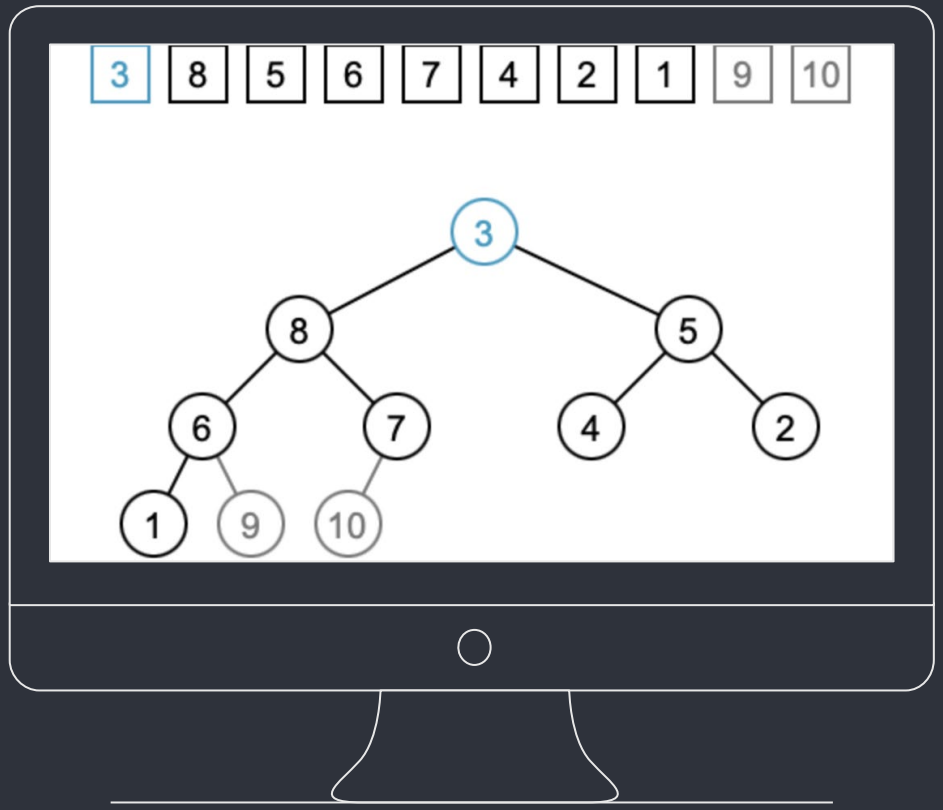


Последний элемент стоит на своем месте, давайте уберем его из кучи?

Пирамида.html

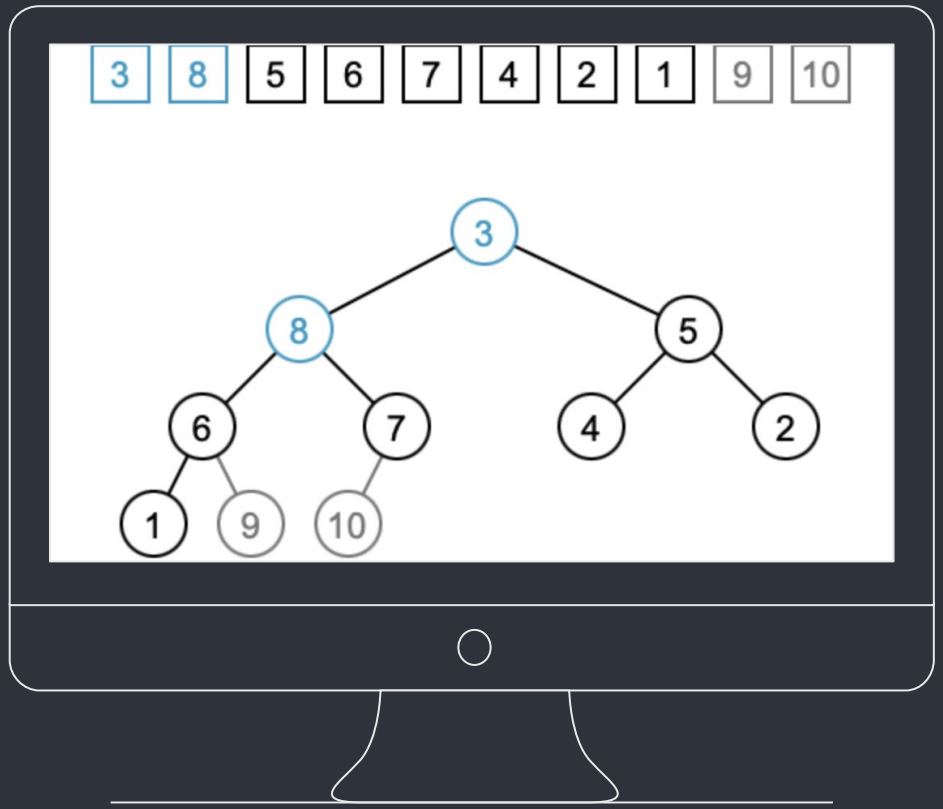
Пирамидальная сортировка.css

```
1
2
3
4  MaxHeap {
5
6
7
8
9
10     heapSize--
11
12
13
14 }
```



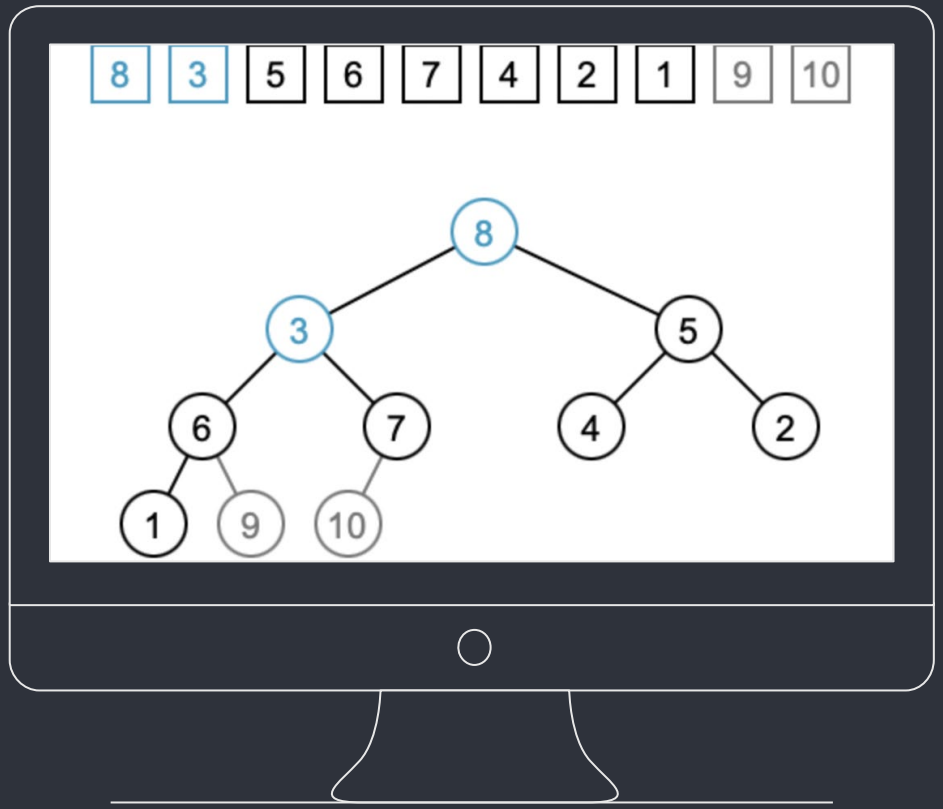
У корневого элемента левое и правое поддерево являются пирамидами - свойство кучи нарушается только в корне

```
Max Heap {
left = 2 * i + 1
right = 2 * i + 2
j = left

if (right < a.heapSize and
a[right] > a[left])
    j = right

if a[j] <= a[i]
    break

swap(a[i], a[j])

}
```



Операция siftDown - просеивание вниз

```
Max Heap {
left = 2 * i + 1
right = 2 * i + 2
j = left

if (right < a.heapSize and
a[right] > a[left])
    j = right

if a[j] <= a[i]
    break

swap(a[i], a[j])

}
```

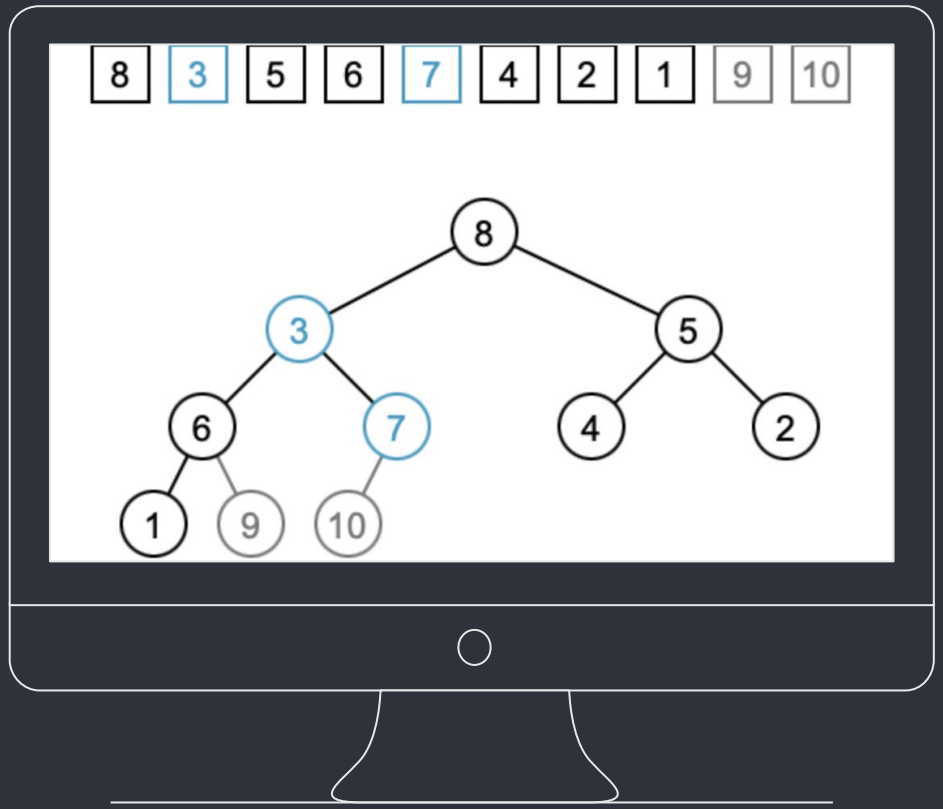


Сравниваем i элемент с большим из дочерних, если i элемент меньше дочернего - меняем их местами

Пирамида.html

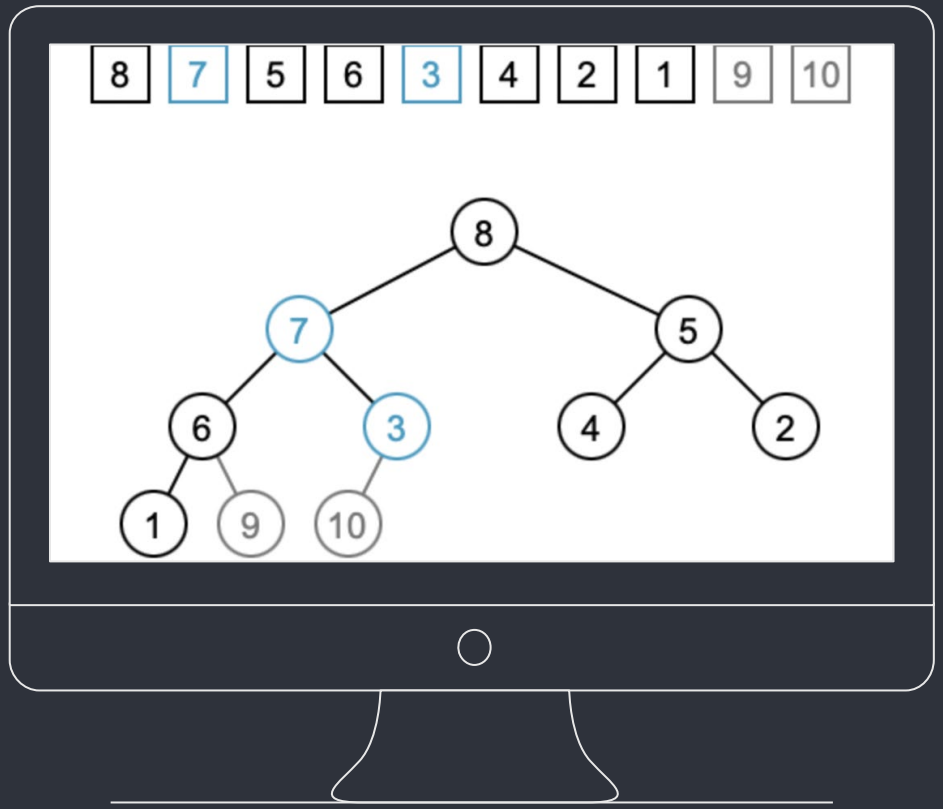
Пирамидальная сортировка.css

```
Max Heap {
left = 2 * i + 1
right = 2 * i + 2
j = left

if (right < a.heapSize and
a[right] > a[left])
    j = right

if a[j] <= a[i]
    break

swap(a[i], a[j])

}
```



Выполняем siftDown до тех пор, пока (i элемент не окажется >= дочерним) или (i элемент не станет листом)

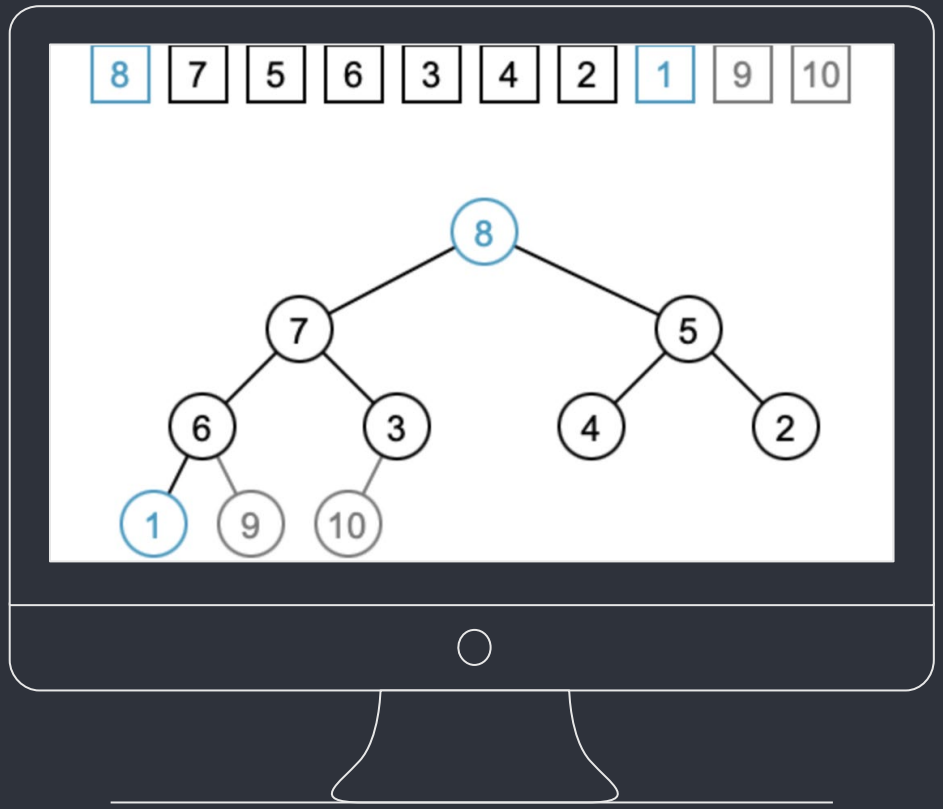
```
Max Heap {
left = 2 * i + 1
right = 2 * i + 2
j = left

if (right < a.heapSize and
a[right] > a[left])
    j = right

if a[j] <= a[i]
    break

swap(a[i], a[j])

}
```



Закончили упражнение - i элемент является листом

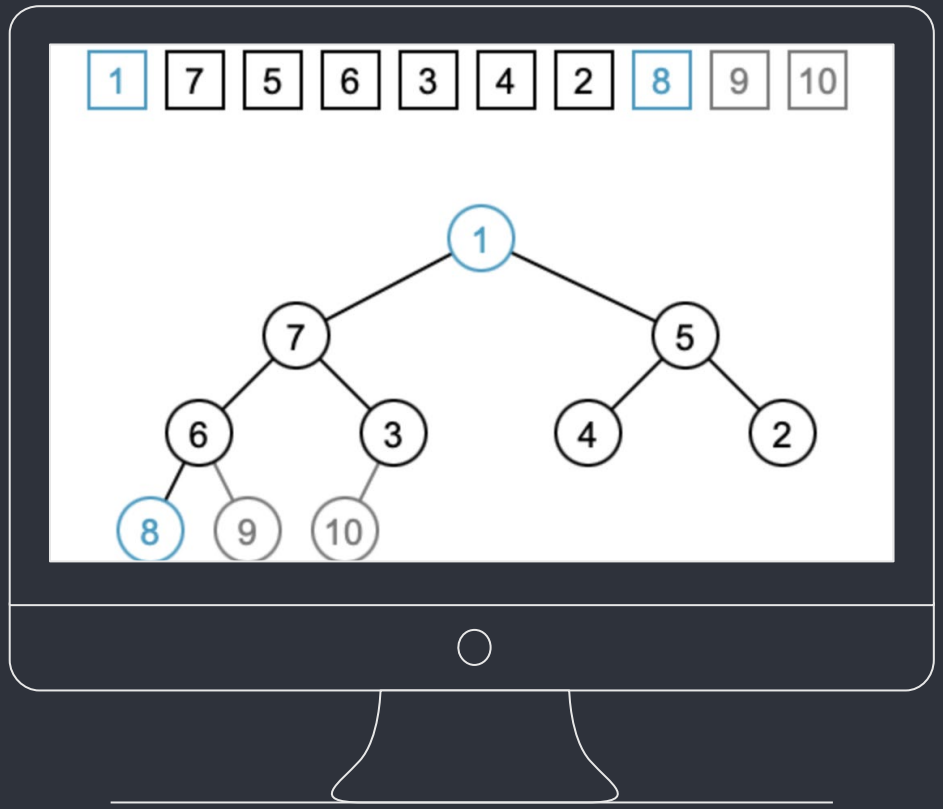
```
1
2
3
4  MaxHeap {
5
6
7
8
9
10
11
12
13
14 }
```

Поменяем первый и последний элемент местами



Swap(a[0], a[heapSize]) - это мы умеем

```
1
2
3
4  Max Heap {
5
6
7
8
9      Поменяли первый и
10     последний элемент
11     местами
12
13
14 }
```

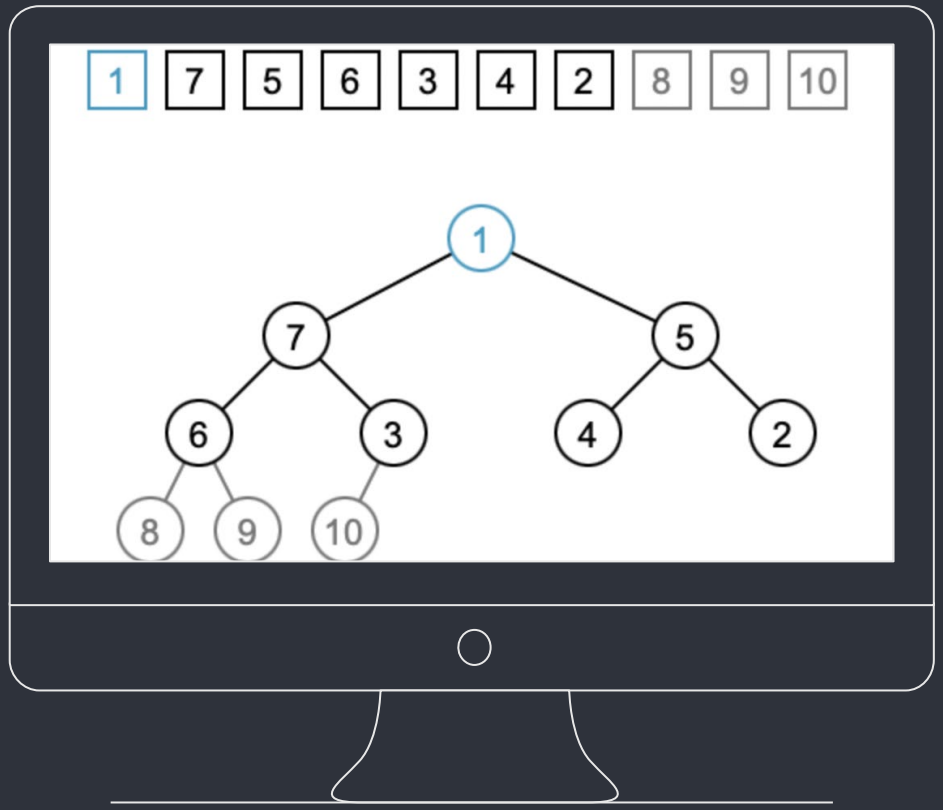


Последний элемент стоит на своем месте, давайте уберем его из кучи?

Пирамида.html

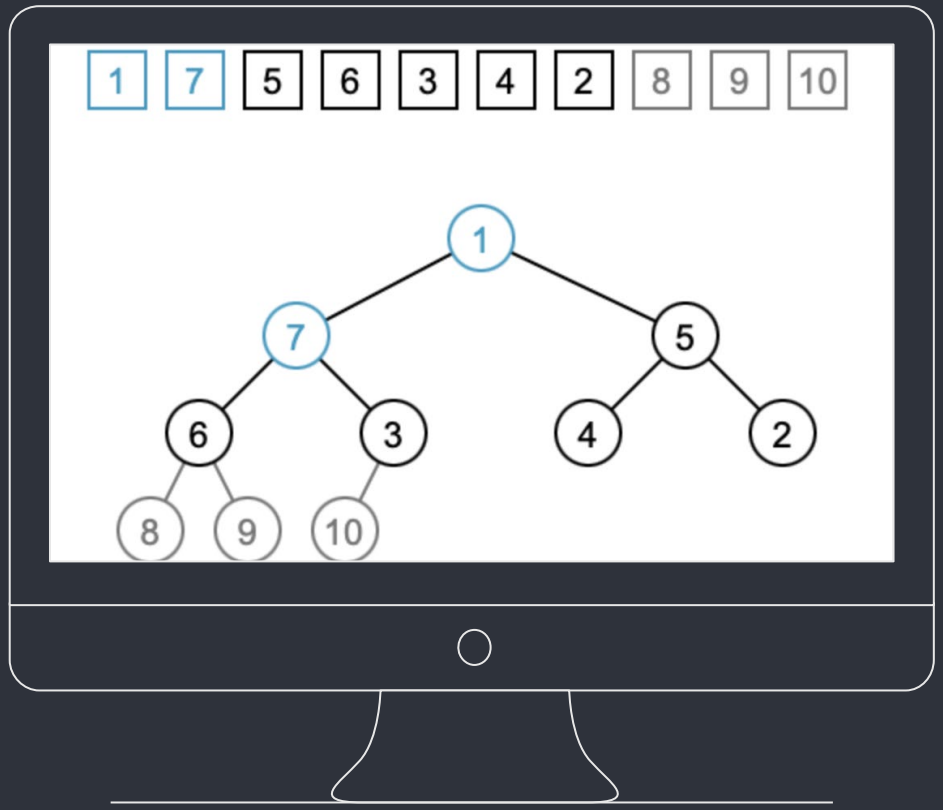
Пирамидальная сортировка.css

```
1
2
3
4  MaxHeap {
5
6
7
8
9
10     heapSize--
11
12
13
14 }
```



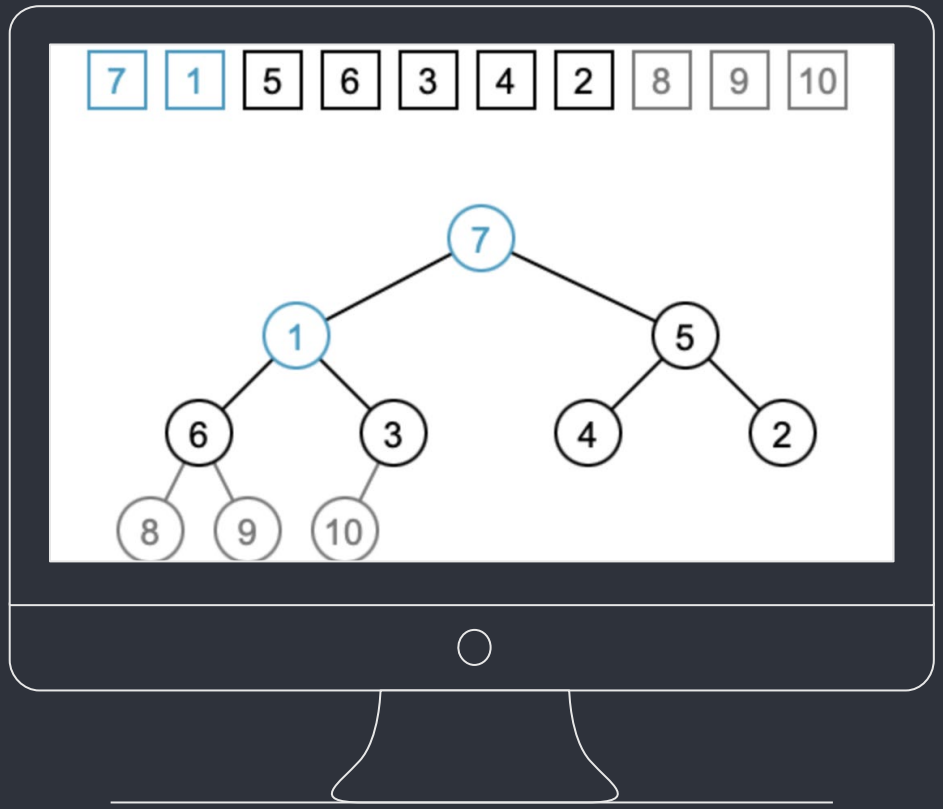
У корневого элемента левое и правое поддерево являются пирамидами - свойство кучи нарушается только в корне

```
Max Heap {
left = 2 * i + 1
right = 2 * i + 2
j = left

if (right < a.heapSize and
a[right] > a[left])
    j = right

if a[j] <= a[i]
    break

swap(a[i], a[j])

}
```



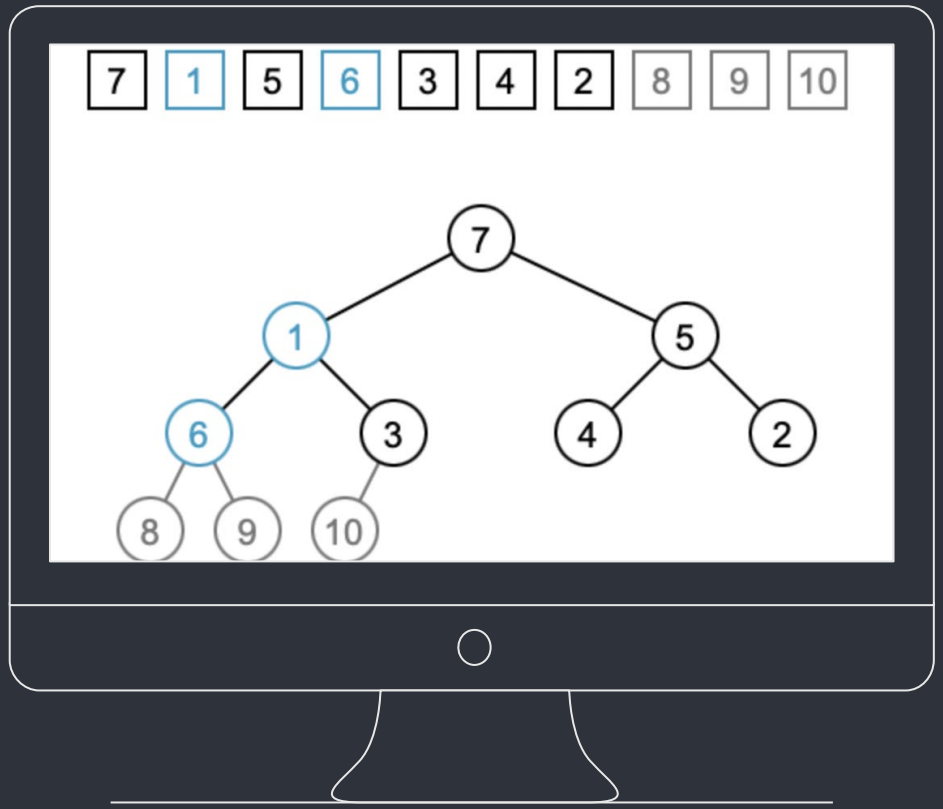
Операция siftDown - просеивание вниз

```
Max Heap {
left = 2 * i + 1
right = 2 * i + 2
j = left

if (right < a.heapSize and
a[right] > a[left])
    j = right

if a[j] <= a[i]
    break

swap(a[i], a[j])

}
```



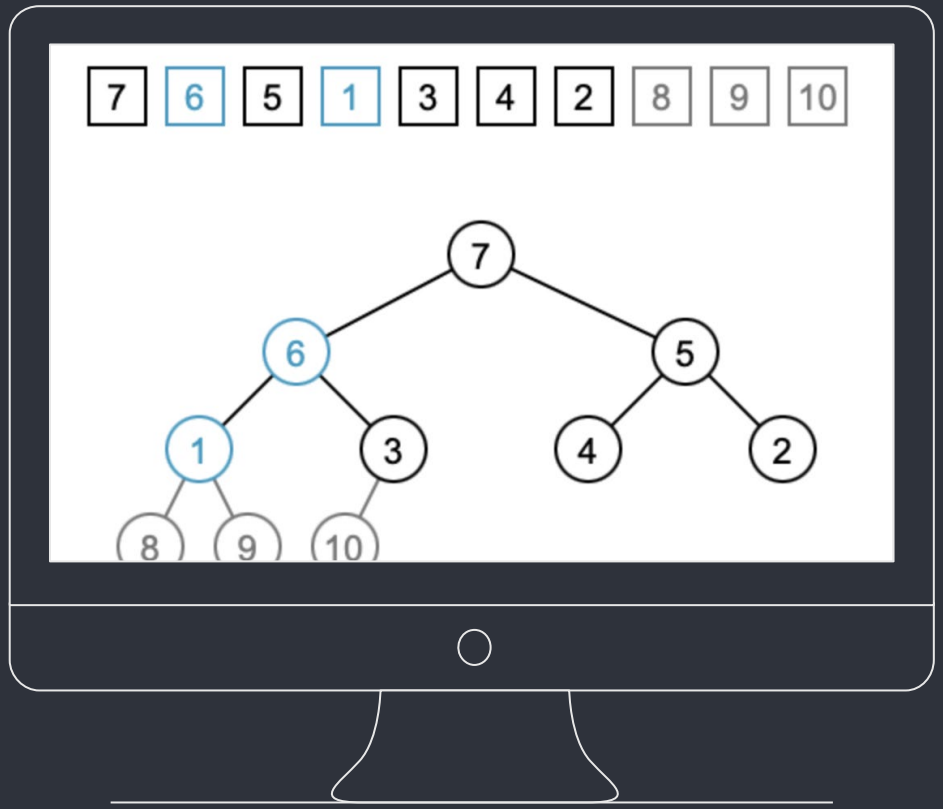
Сравниваем i элемент с большим из дочерних, если i элемент меньше дочернего - меняем их местами

```
1
2  Max Heap {
3  left = 2 * i + 1
4  right = 2 * i + 2
5  j = left
6
7  if (right < a.heapSize and
   a[right] > a[left])
8      j = right
9
10 if a[j] <= a[i]
11     break
12
13 swap(a[i], a[j])
14
   }
```



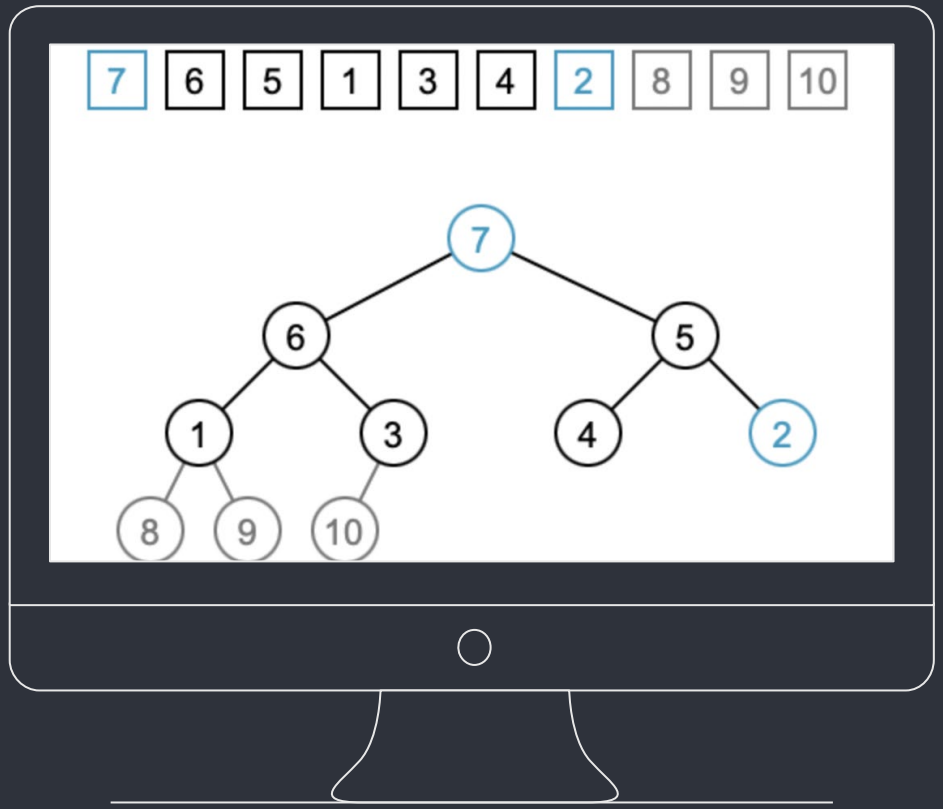
Выполняем siftDown до тех пор, пока (i элемент не окажется >= дочерним) или (i элемент не станет листом)

```
Max Heap {
left = 2 * i + 1
right = 2 * i + 2
j = left

if (right < a.heapSize and
a[right] > a[left])
    j = right

if a[j] <= a[i]
    break

swap(a[i], a[j])

}
```



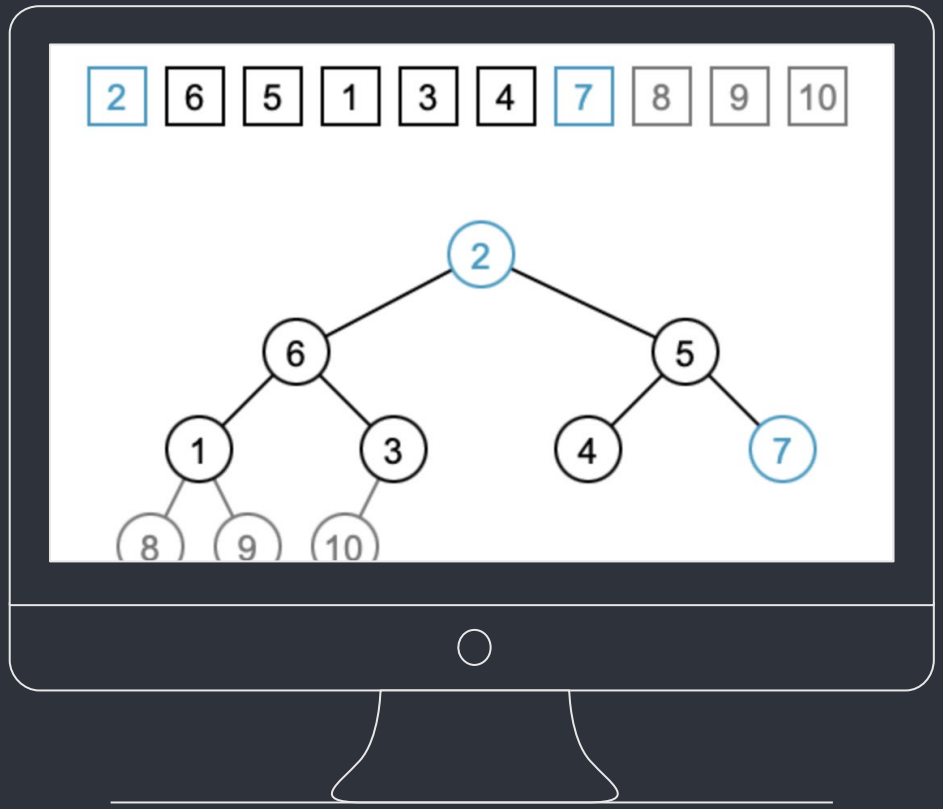
Закончили упражнение - i элемент является листом

```
1
2
3
4  MaxHeap {
5
6
7
8
9      Поменяем первый и
10     последний элемент
11     местами
12
13
14 }
```



Swap(a[0], a[heapSize]) - это мы умеем

```
1
2
3
4  Max Heap {
5
6
7
8
9      Поменяли первый и
10     последний элемент
11     местами
12
13
14 }
```

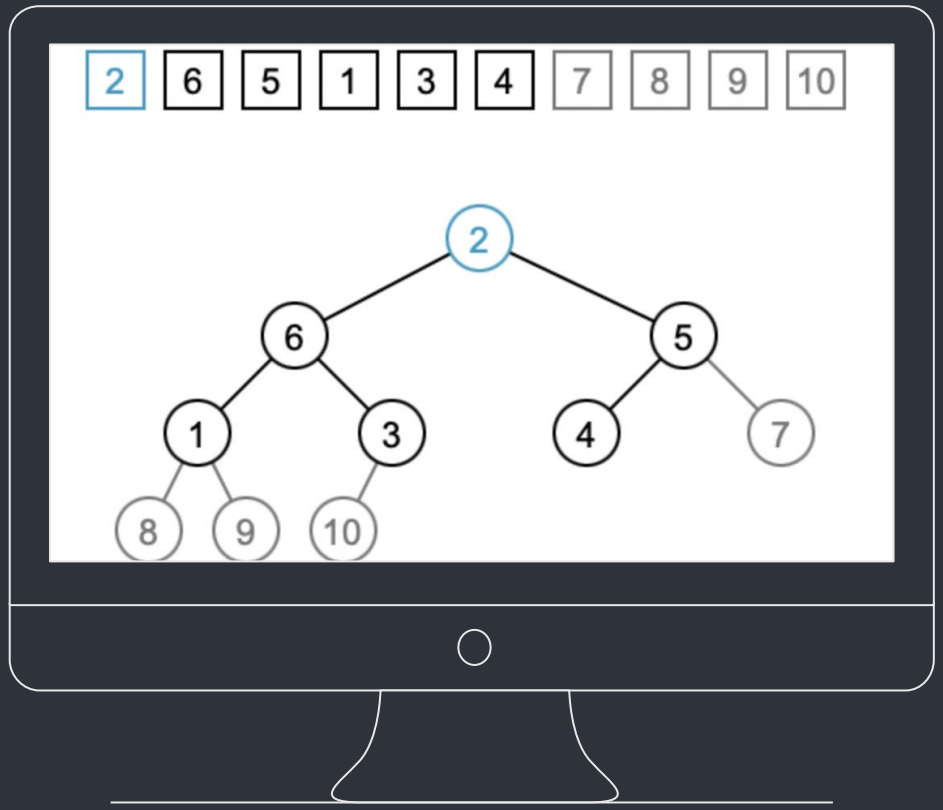


Последний элемент стоит на своем месте, давайте уберем его из кучи?

Пирамида.html

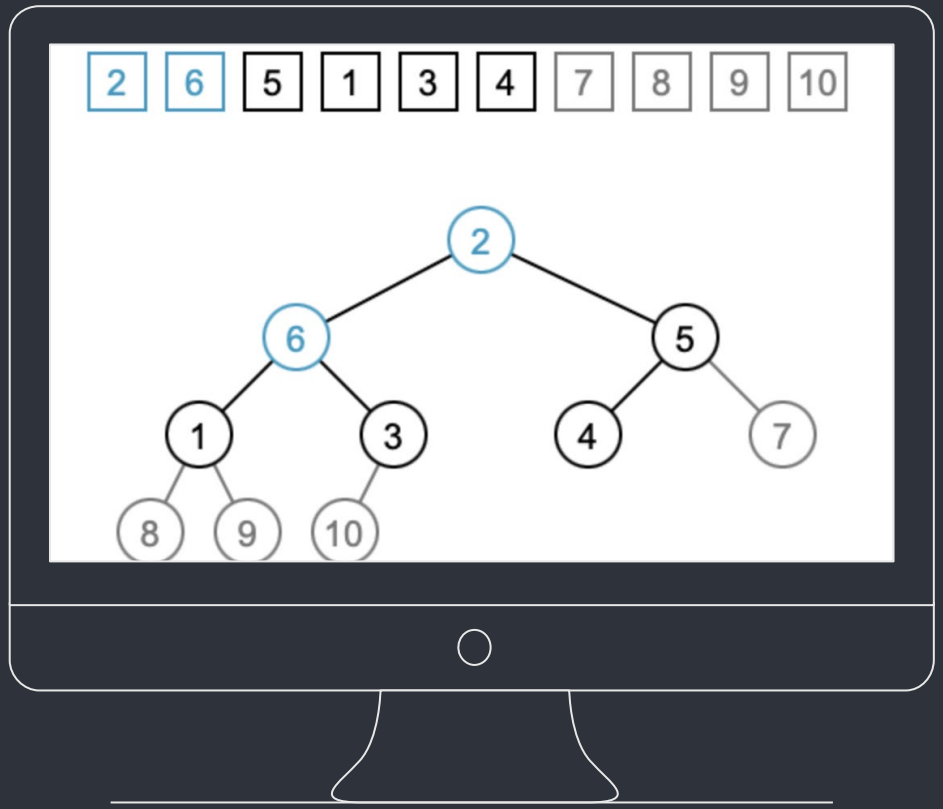
Пирамидальная сортировка.css

```
1
2
3
4  MaxHeap {
5
6
7
8
9
10     heapSize--
11
12
13
14 }
```



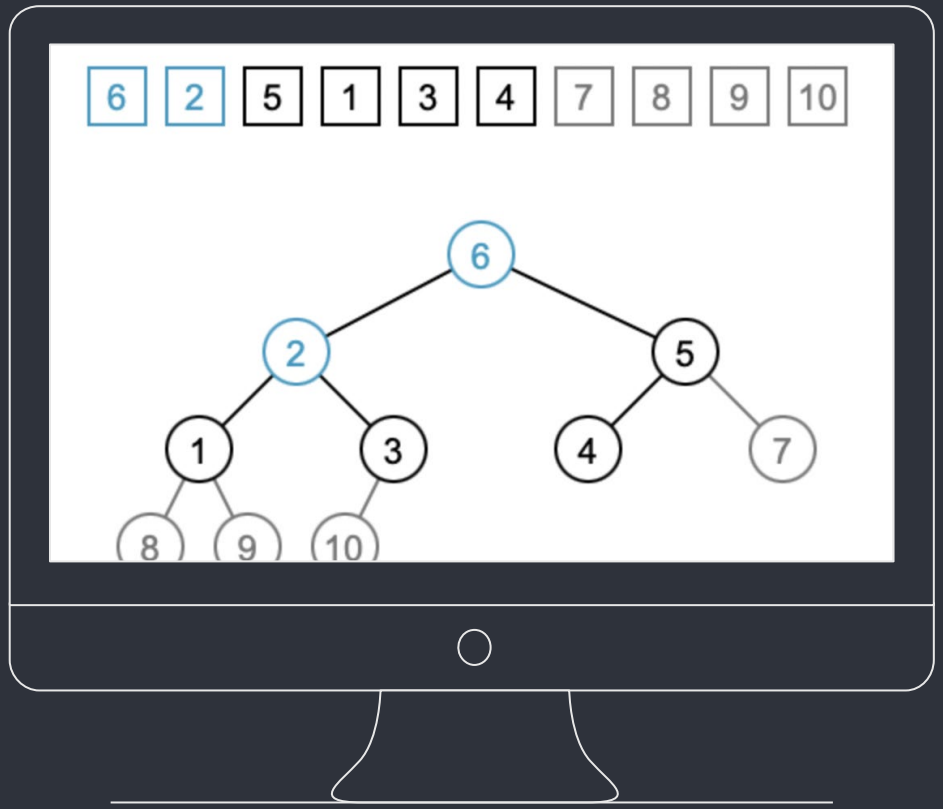
У корневого элемента левое и правое поддерево являются пирамидами - свойство кучи нарушается только в корне

```
Max Heap {
left = 2 * i + 1
right = 2 * i + 2
j = left

if (right < a.heapSize and
a[right] > a[left])
    j = right

if a[j] <= a[i]
    break

swap(a[i], a[j])

}
```



Операция siftDown - просеивание вниз

```
Max Heap {
left = 2 * i + 1
right = 2 * i + 2
j = left

if (right < a.heapSize and
a[right] > a[left])
    j = right

if a[j] <= a[i]
    break

swap(a[i], a[j])

}
```

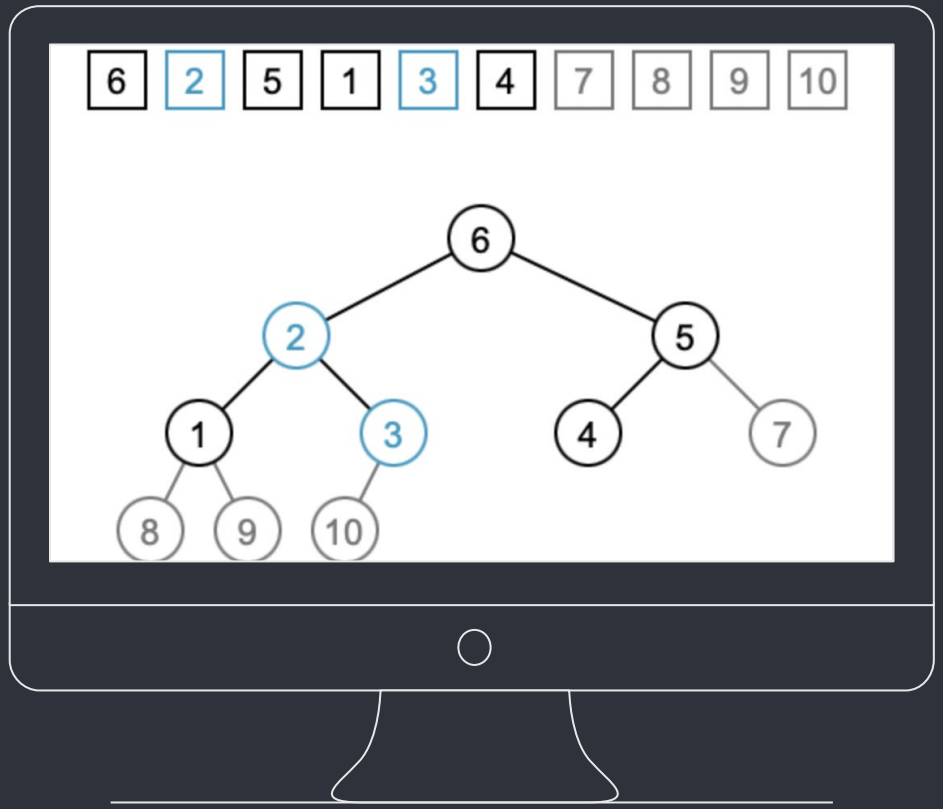


Сравниваем i элемент с большим из дочерних, если i элемент меньше дочернего - меняем их местами

Пирамида.html

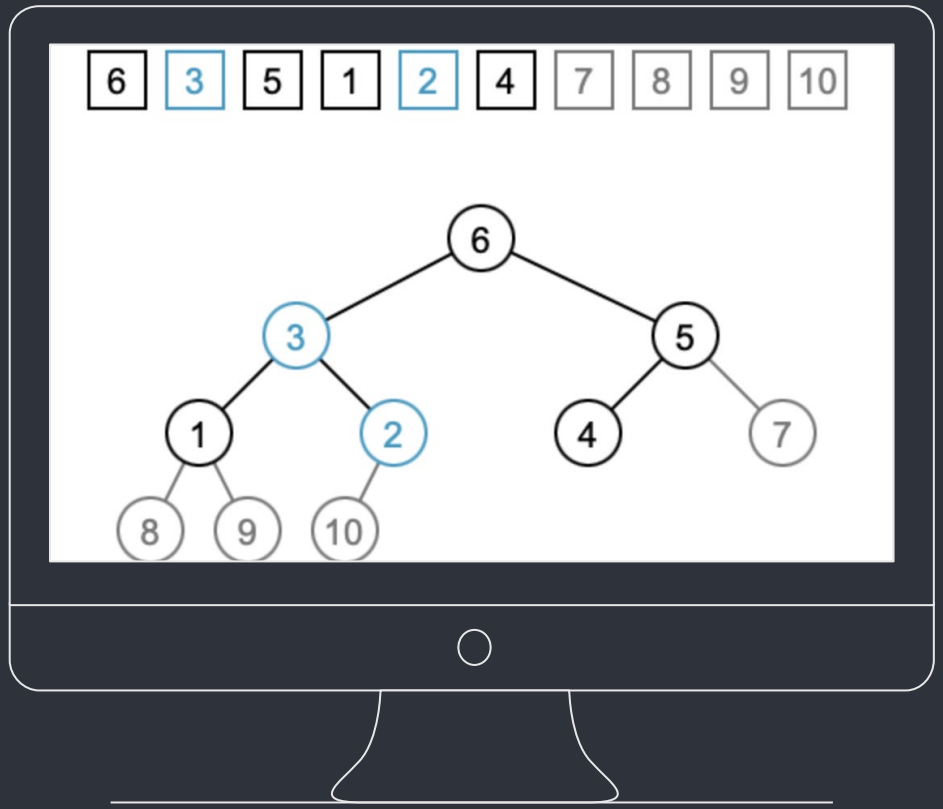
Пирамидальная сортировка.css

```
Max Heap {
left = 2 * i + 1
right = 2 * i + 2
j = left

if (right < a.heapSize and
a[right] > a[left])
    j = right

if a[j] <= a[i]
    break

swap(a[i], a[j])

}
```



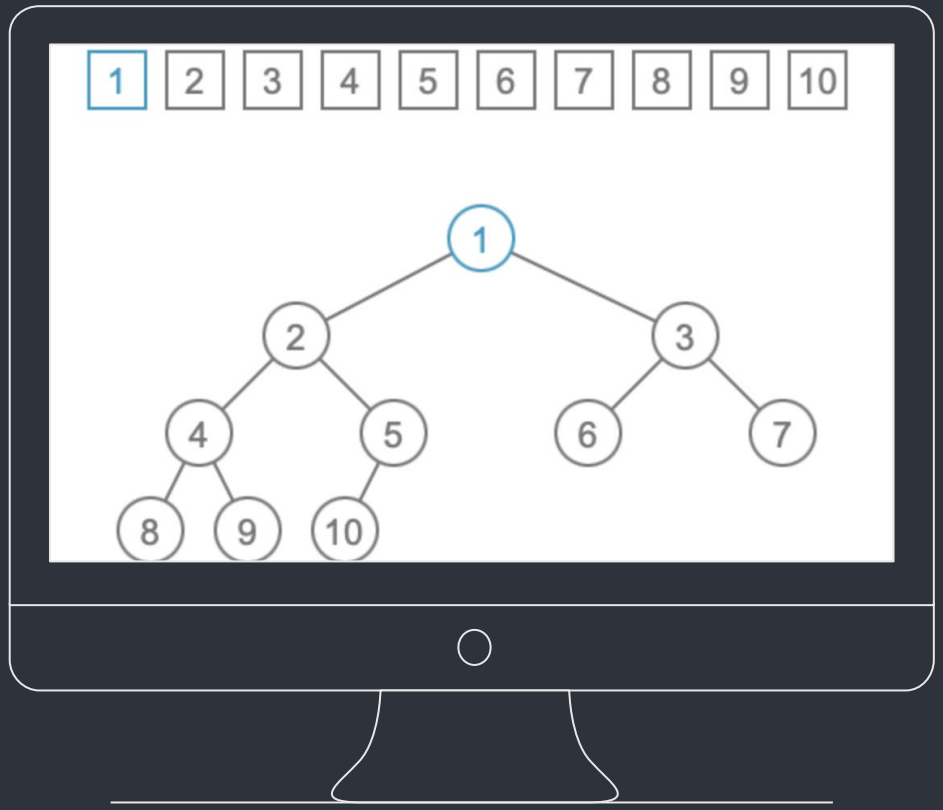
Выполняем siftDown до тех пор, пока (i элемент не окажется >= дочерним) или (i элемент не станет листом)

```
Max Heap {
left = 2 * i + 1
right = 2 * i + 2
j = left

if (right < a.heapSize and
a[right] > a[left])
    j = right

if a[j] <= a[i]
    break

swap(a[i], a[j])

}
```



Закончили упражнение - i элемент является листом

```
1
2
3
4  MaxHeap {
5
6
7
8      Выполнив n раз
9      описанные итерации,
10     получим
11     отсортированный массив
12
13
14 }
```



Асимптотика: n раз выполнили siftDown, который работает за O(logn)
=> $O(n*\log_2 n)$

Пирамида.html

Устойчивость сортировки.css

1
2
3
4
5
6 < “Правильно ли я понимаю, что
7 пирамидальная сортировка не является
8 устойчивой?” >
9
10 — Кто-то самый внимательный в аудитории
11
12
13
14

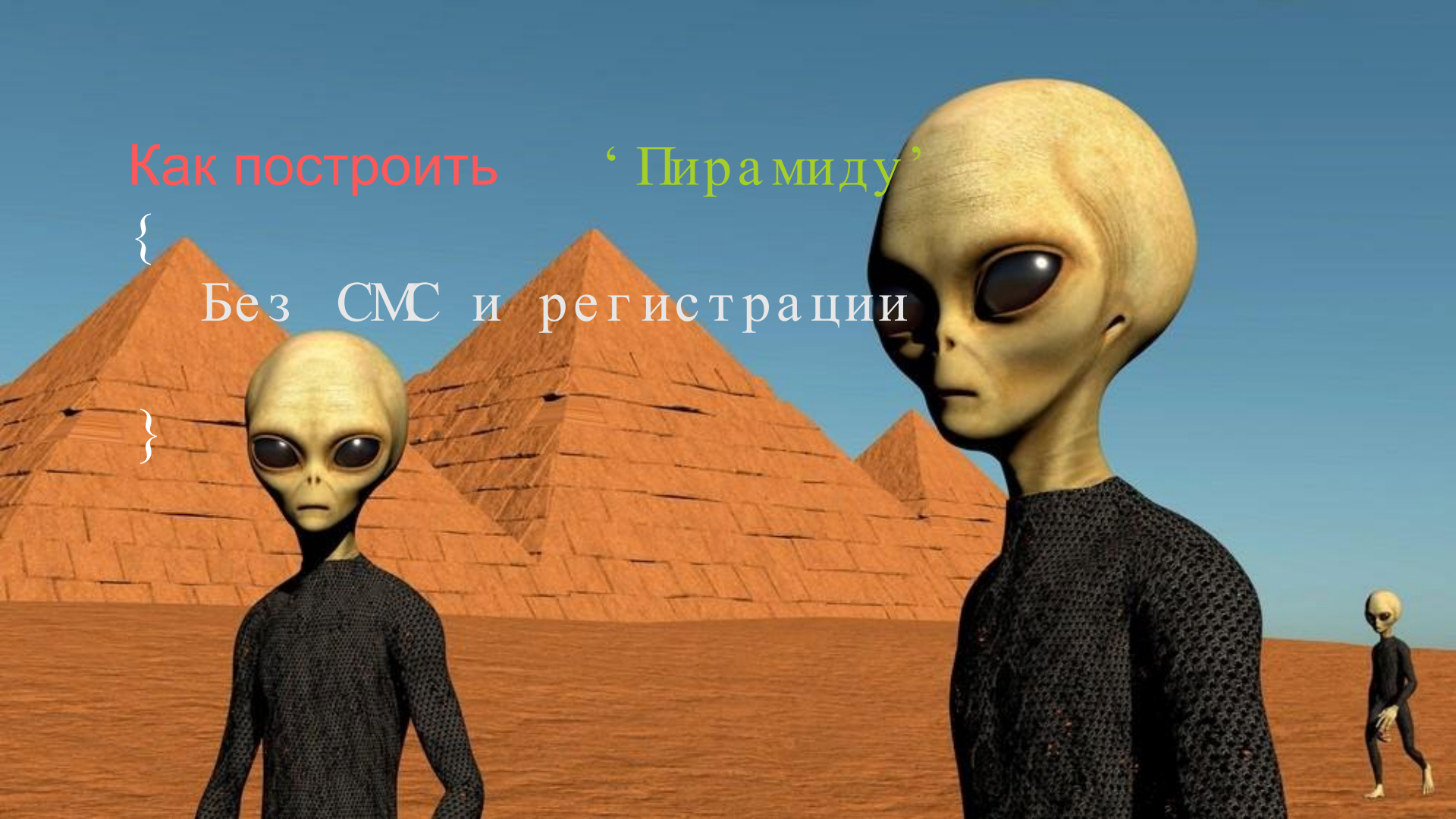
Когда разбираете новую сортировку, всегда задумывайтесь об устойчивости

Пирамида.html

Построение пирамиды.css

1
2
3
4
5 < “Вау, крутая сортировка. Новая
6 структура данных, похожая на
7 соревнования по футболу. А как
8 построить то это чудо (пирамиду)?” >
9
10 —Кто-то самый впечатлительный в аудитории
11
12
13
14

Пирамидальная сортировка => ломать - не строить


Как построить ‘Пирамиду’
{
Без СМС и регистрации
}

Какими ‘свойствами’ обладает ‘пустота’?

{

< Если я заведу пустой массив на n элементов, могу ли я его считать пустой кучей? >

}

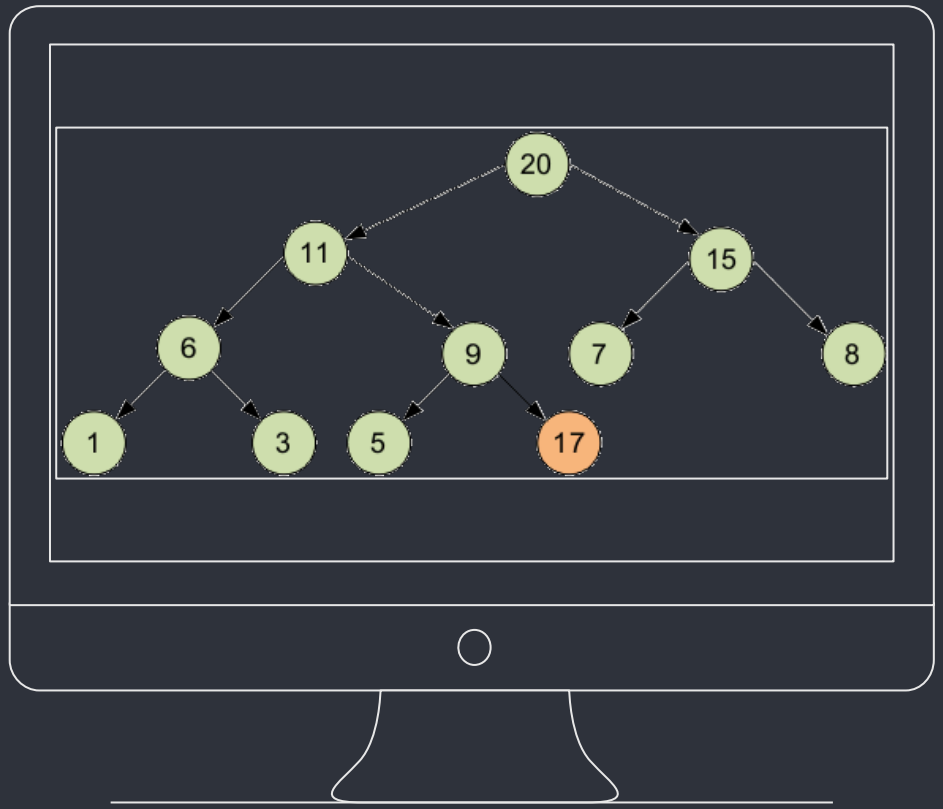
В таком случае {

< Построение кучи из n элементов = последовательному применению операции insert >

}

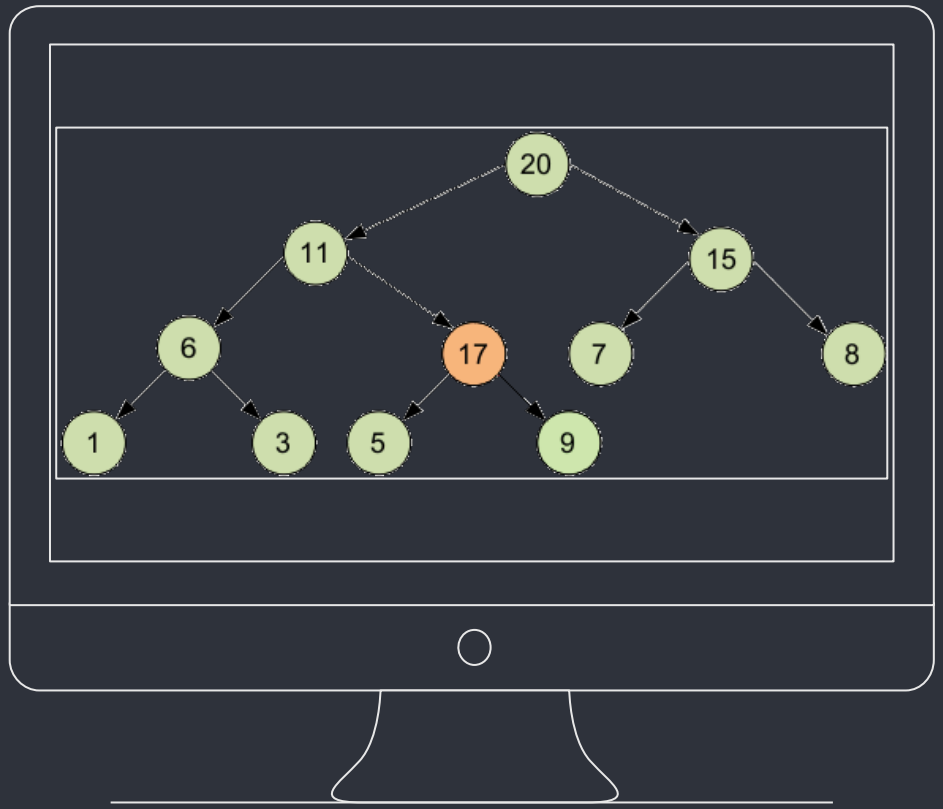
Звучит просто. Осталось разобраться, как работает операция insert.

```
insert {

    Новый элемент
    добавляется на
    последнее место в
    массиве, то есть
    позицию с максимальным
    индексом = heapSize.

}
```



Возможно, что при этом будет нарушено основное свойство кучи, так как новый элемент может быть больше родителя.

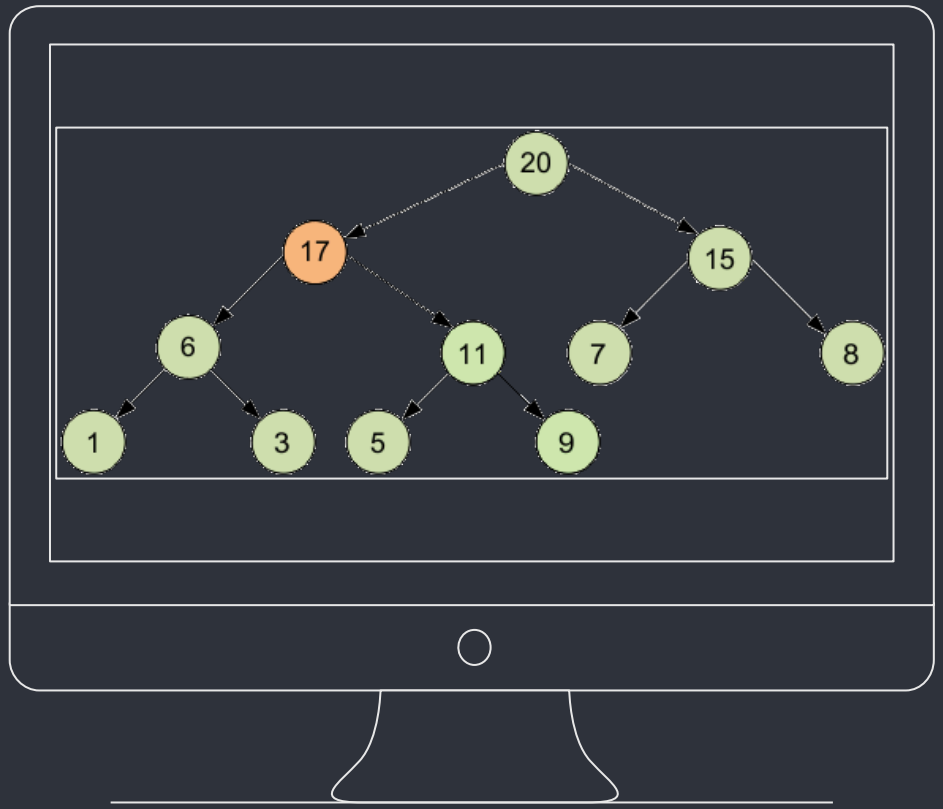
```
insert {
    В таком случае новый элемент «поднимается» на один уровень (менять с вершиной-родителем) до тех пор, пока не будет соблюдено основное свойство кучи.
}
```



Сложность алгоритма не превышает высоты двоичной кучи (так как количество «подъемов» не больше высоты дерева), то есть равна $\log_2 n$.

Псевдокод

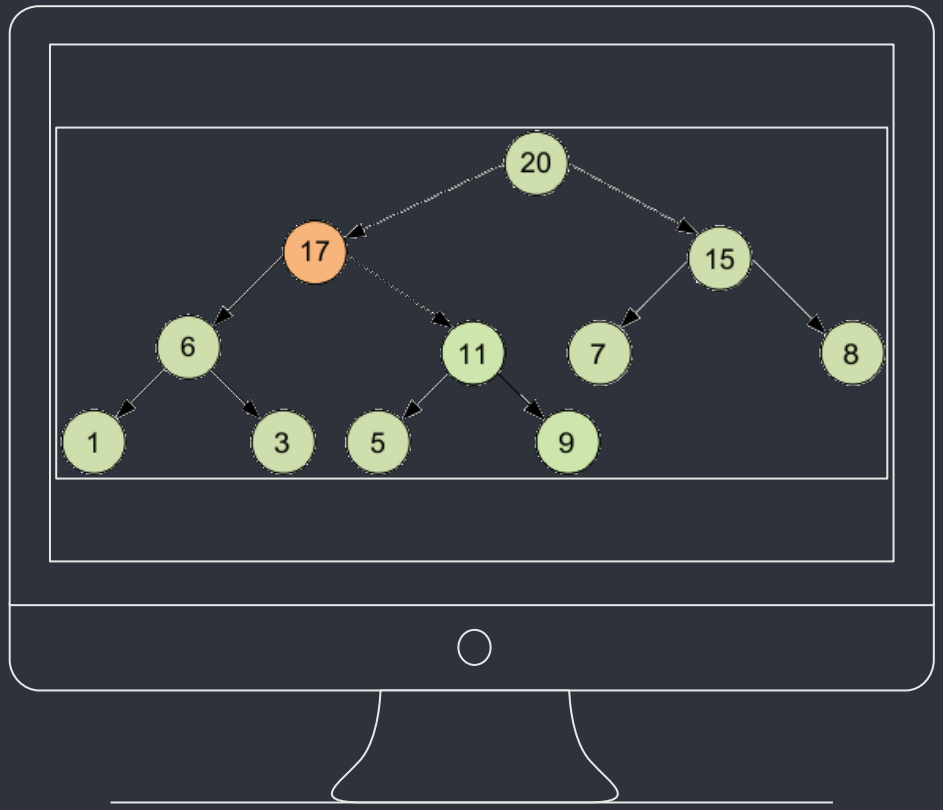
```
siftUp {

while a[i] < a[(i - 1) / 2]
    swap(a[i], a[(i - 1) / 2])
    i = (i - 1) / 2

}
```



Сложность алгоритма не превышает высоты двоичной кучи (так как количество «подъемов» не больше высоты дерева), то есть равна log2 N.

Псевдокод

```
insert {


a.heapSize = a.heapSize + 1
a[a.heapSize - 1] = key
siftUp(a.heapSize - 1)



}
```



Увеличили размер кучи на 1 => добавили элемент в конец => просеяли вверх.

Пирамида.html

Удаление i элемента из кучи.css

1
2
3
4
5
6 < “А что если мне нужно удалить i
7 элемент из кучи?” >
8
9
10 —Кто-то самый любознательный в аудитории
11
12
13
14

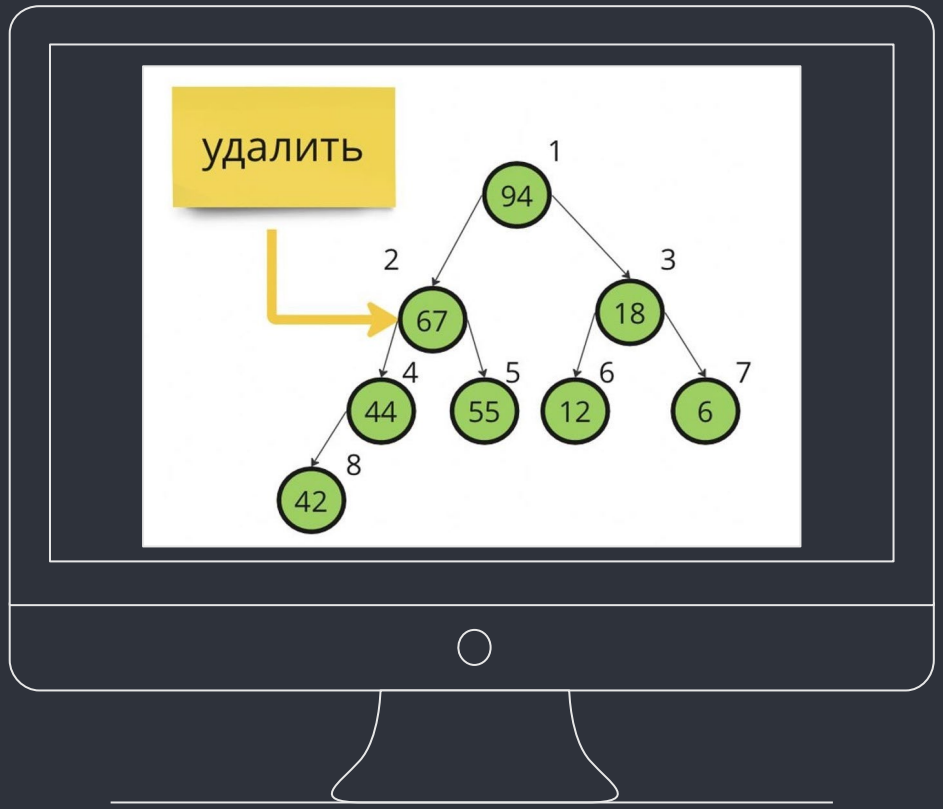
Хочешь что-то удалить => добавь в конец

Пирамида.html

Удаление i элемента из кучи.css

```
delete(i) {

}
```

Давайте поменяем удаляемый и последний элементы местами

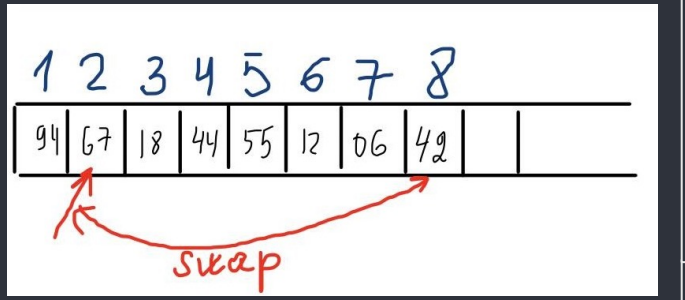


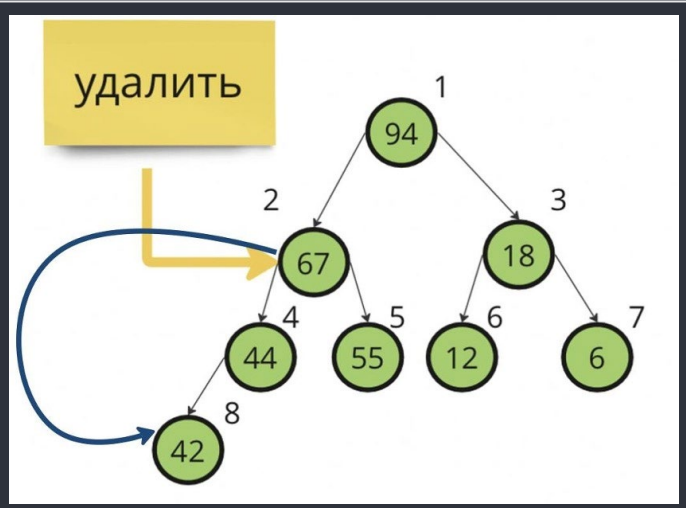
Кажется мы так поступали, когда удаляли максимальный элемент

Пирамида.html

Удаление i элемента из кучи.css

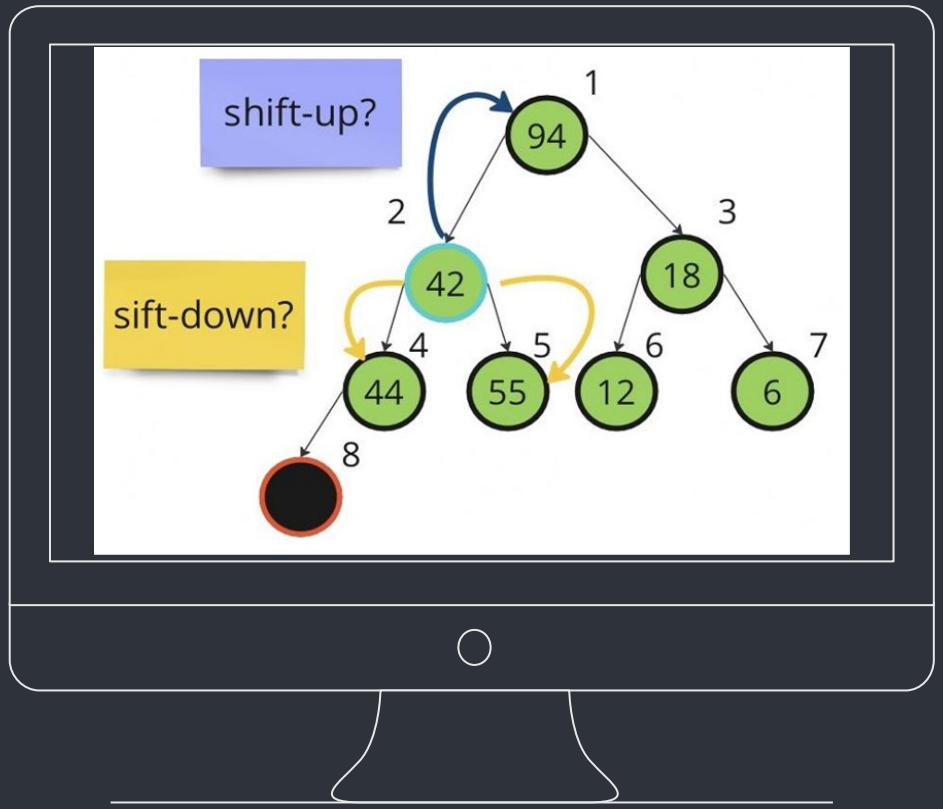
```
delete(i) {

}
```





Если проводить аналогию, то дальше необходимо уменьшить размер кучи и сделать просеивание вниз

```
delete(i) {

Элемент нарушающий свойство кучи находится не в корне => мы не знаем, в какую сторону необходимо просеять i элемент

}
```



Так давайте попробуем просеять последовательно вверх и вниз

Пирамида.html

Удаление i элемента из кучи.css

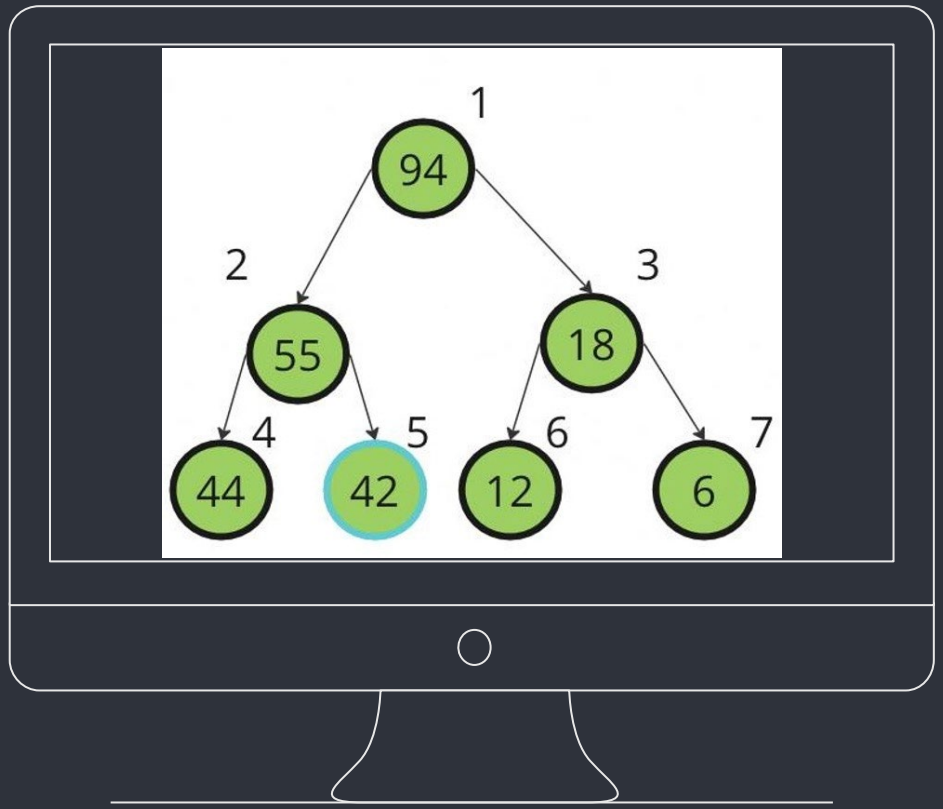
```
delete(i) {

    swap(a[i], a[heapsize]
    heapsize--
    siftUp(a[i])
    siftDown(a[i])

}
```

Поменять удаляемый и последний элементы местами => уменьшить размер кучи => попробовать просеять вверх => попробовать просеять вниз.

Пирамида.html

Удаление i элемента из кучи.css

```
delete(i) {

}
```



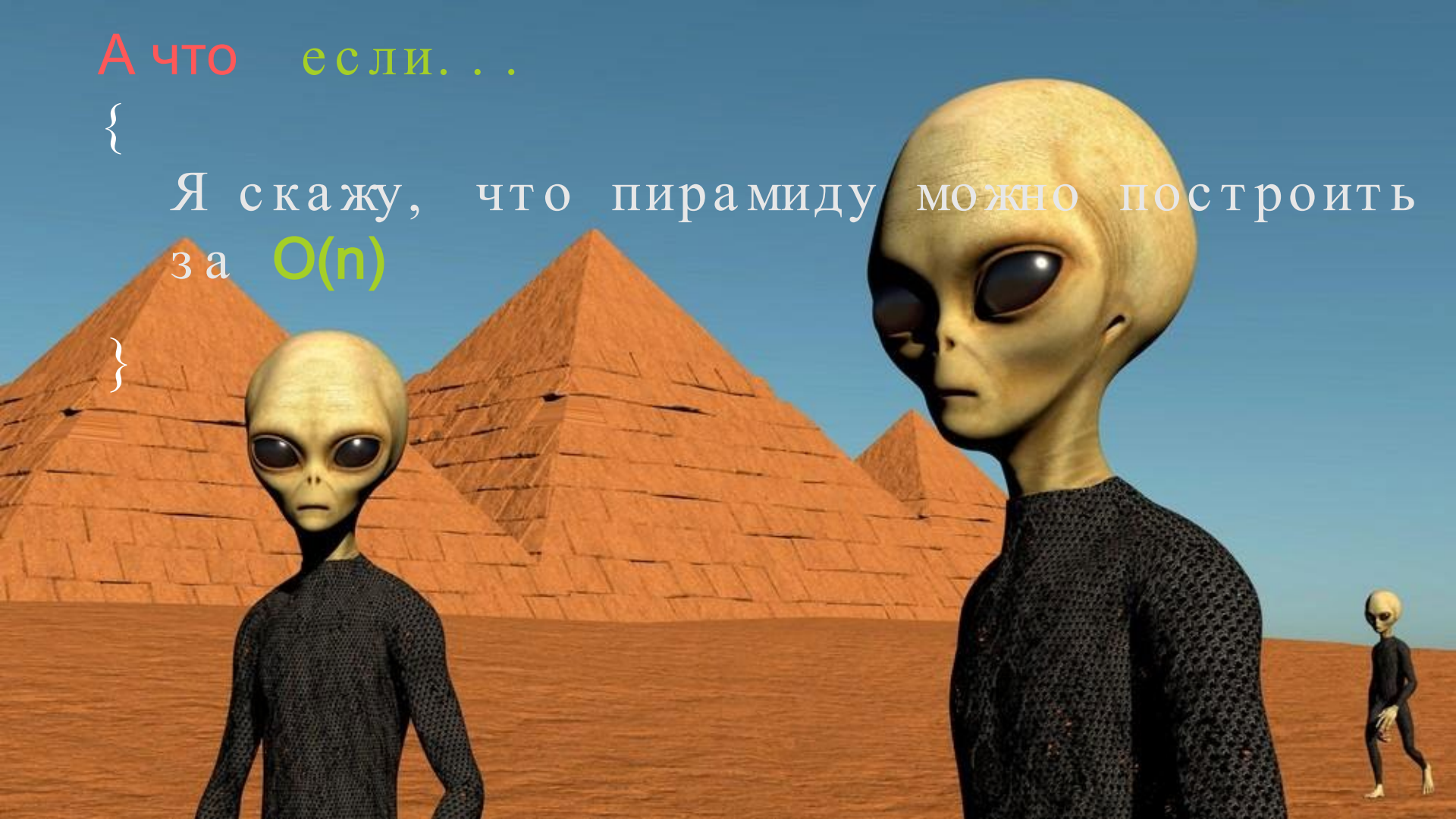
Неужели мы теперь знаем о пирамиде всё...

А что если...

{

Я скажу, что пирамиду можно построить за O(n)

}



Пирамида.html

Эффективное построение пирамиды.css

1
2
3
4
5
6 < “Но зачем строить пирамиду за O(n),
7 если пирамидальная сортировка все
8 равно работает за $O(n*\log_2 n)$?” >
9
10 — Кто-то немного ленивый в аудитории
11
12
13
14

И это хороший вопрос!

Пирамида </1> {

Пирамида - это структура данных, которая используется в различных алгоритмах:

- Пирамидальная сортировка
- Реализация очереди с приоритетом
- Алгоритм Дейкстры
- Алгоритм Прима
- Алгоритм Хаффмана
- Сортировка потоков
- Алгоритмы обработки данных в реальном времени
- Алгоритмы обработки событий
- Алгоритмы геометрии
- Алгоритмы поиска минимального покрытия

}

Пирамидальная сортировка </2> {

Пирамидальная сортировка - один из алгоритмов, который использует структуру данных “Пирамида”

}

Пирамида.html

Эффективное построение пирамиды.css

1
2
3
4
5
6 < “Понятно. Показывайте построение
7 пирамиды за O(n).” >
8
9
10 —Кто-то нетерпеливый в аудитории
11
12
13
14

На самом деле преподавателя очень радует ваша любознательность

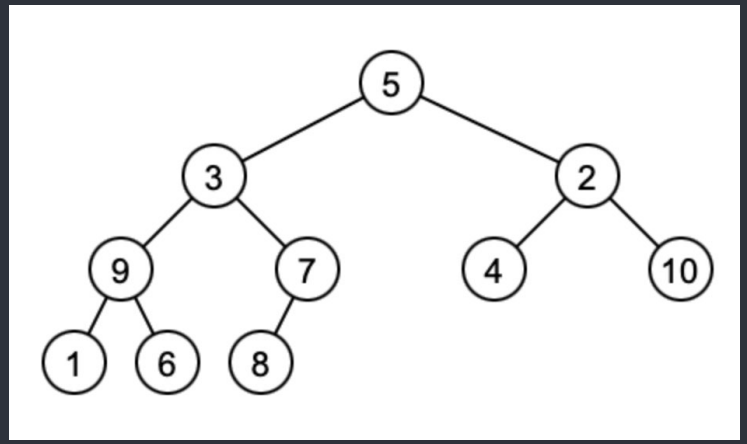
Когда на коллоквиуме попросили
построить пирамиду за O(n)

Является ли деревом? {

}

Да какое же это дерево? Это массив! Или нет...

1
2
3
4
5 А вот так? {
6
7
8
9
10
11
12
13
14 }



Хорошо, убедили, это дерево! Но это не пирамида!

Вспоминаем нашу ‘мантру’ {

01 Каждый охотник желает знать...

< Нет, это не то, что нам нужно >

02 Каждое поддерево бинарного дерева...

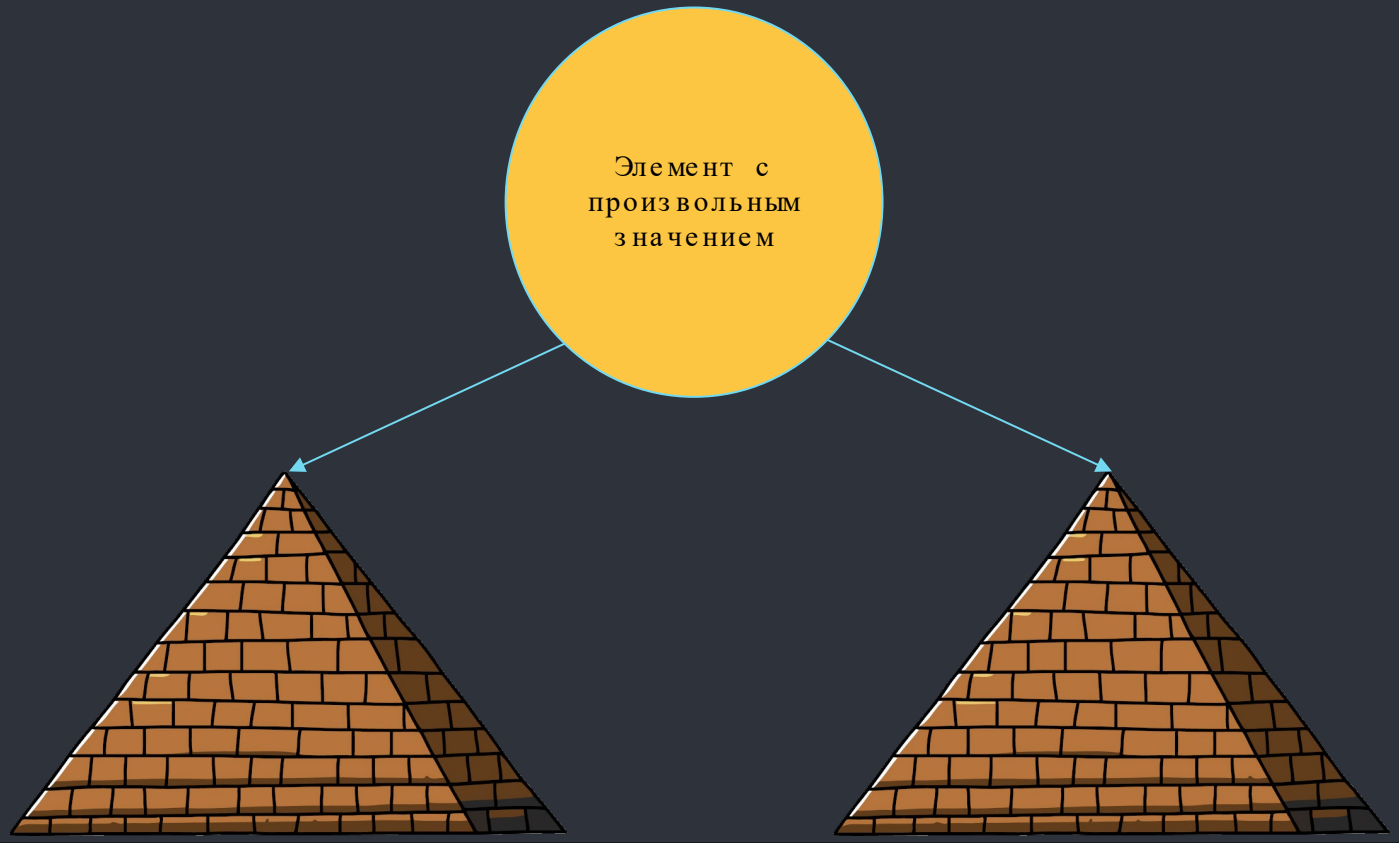
< является бинарным деревом >

03 Листовые элементы...

< любого бинарного дерева - бинарные деревья, удовлетворяющие свойству кучи <=> пирамида >

}

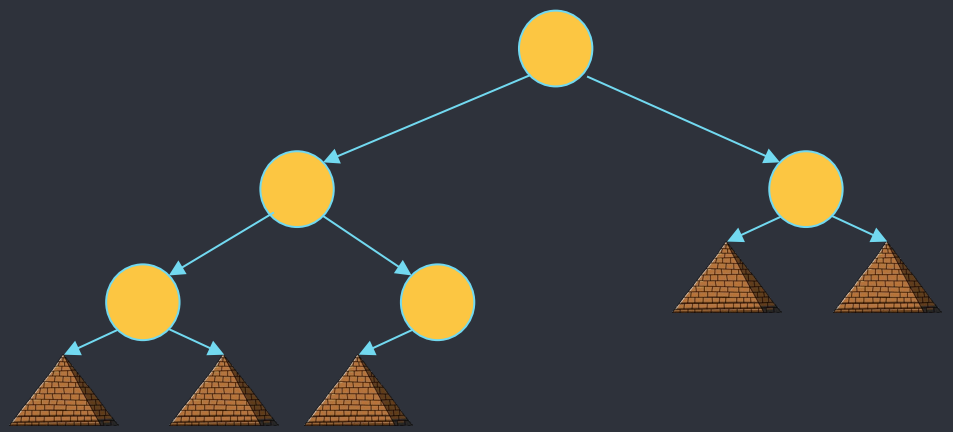
1
2
3
4
5
6
7
8
9
10
11
12
13
14



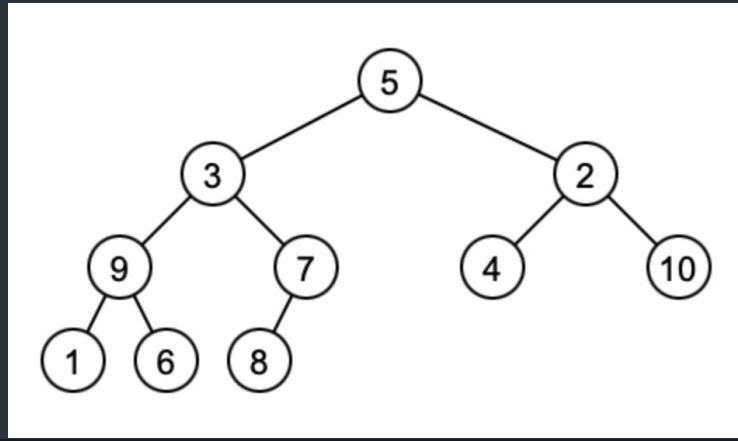
Достаточно применить siftDown, чтобы восстановить свойство кучи

Пирамида.html

Эффективное построение пирамиды.css



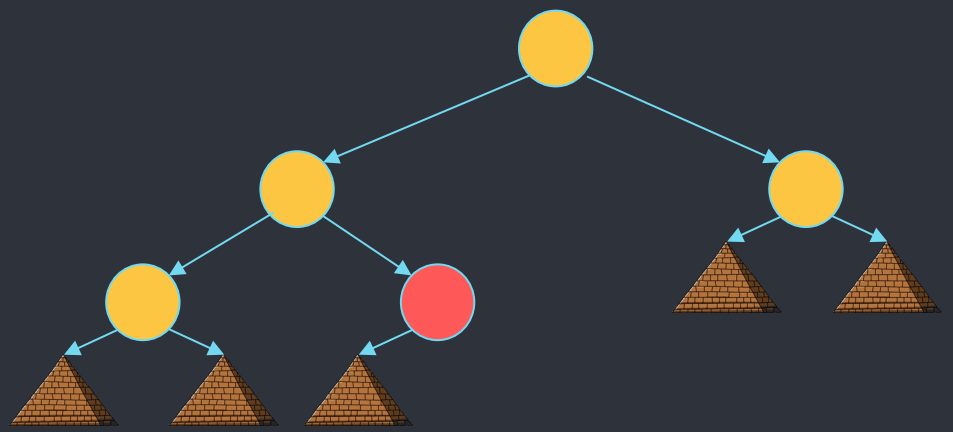
Запустим функцию siftDown для элементов от n/2 до 0



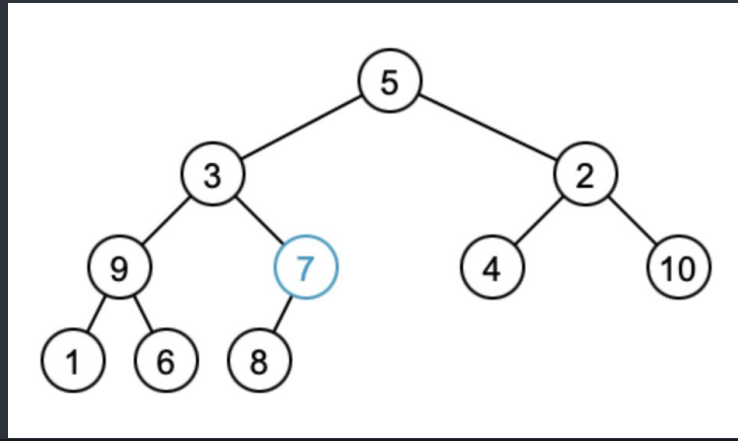
Запустим siftDown от первого элемента имеющего потомков, до корня

Пирамида.html

Эффективное построение пирамиды.css



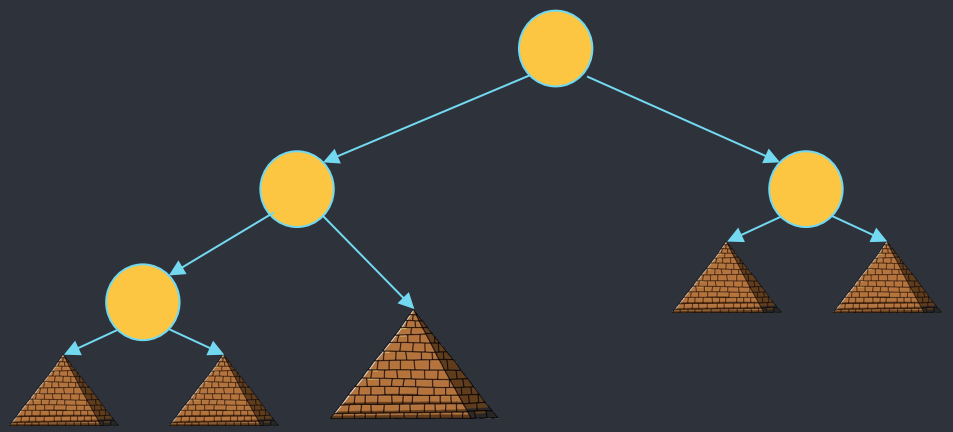
Запустим функцию siftDown для элементов от n/2 до 0



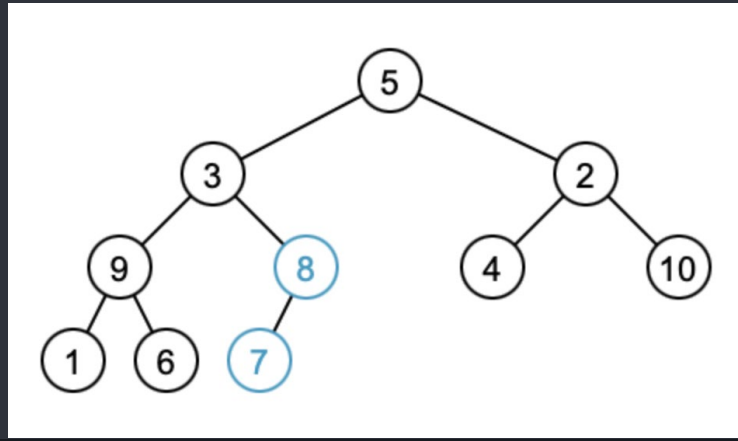
Запустим siftDown от первого элемента имеющего потомков, до корня

Пирамида.html

Эффективное построение пирамиды.css



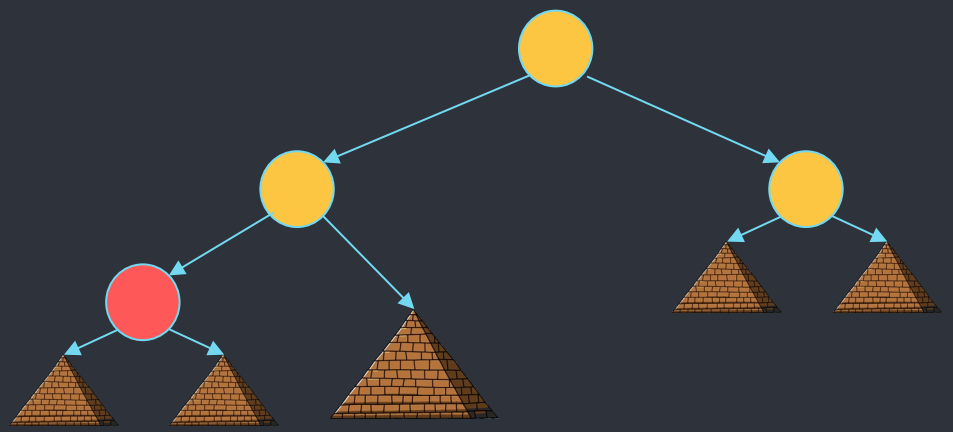
Запустим функцию siftDown для элементов от n/2 до 0



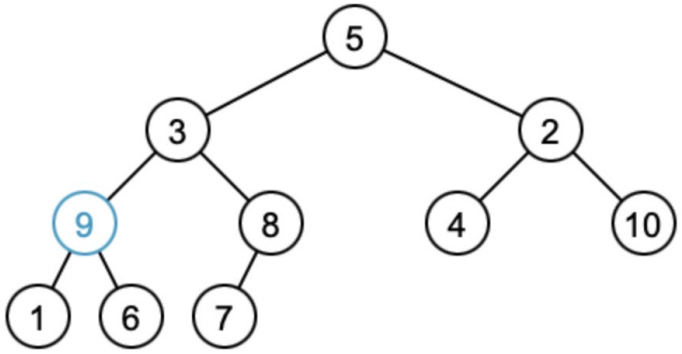
Запустим siftDown от первого элемента имеющего потомков, до корня

Пирамида.html

Эффективное построение пирамиды.css



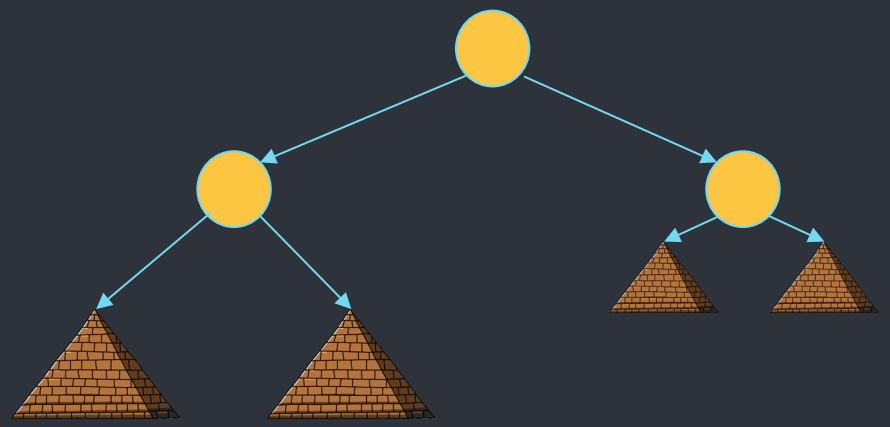
Запустим функцию siftDown для элементов от n/2 до 0



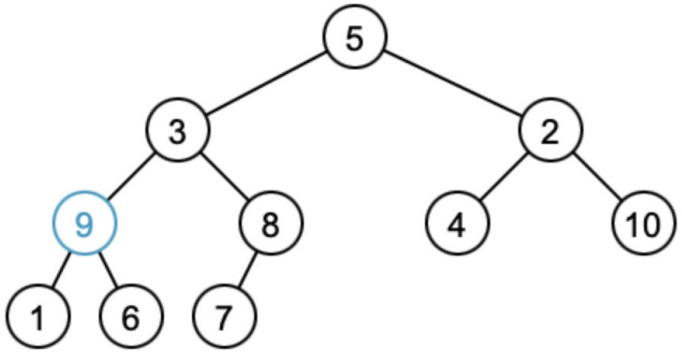
Запустим siftDown от первого элемента имеющего потомков, до корня

Пирамида.html

Эффективное построение пирамиды.css



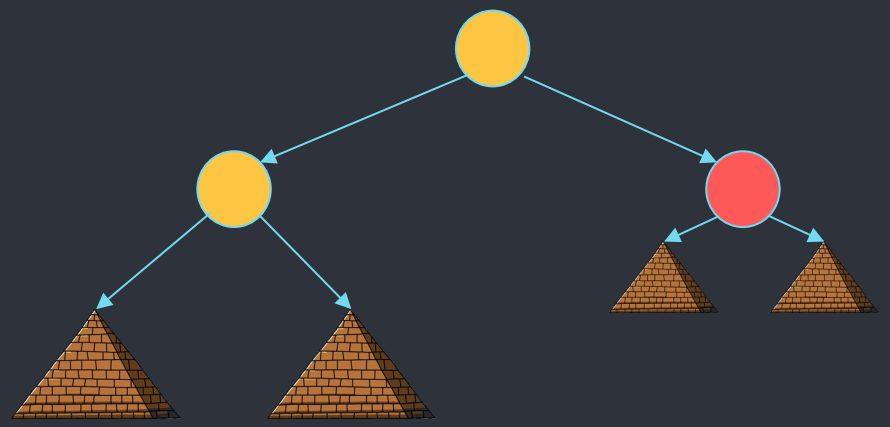
Запустим функцию siftDown для элементов от n/2 до 0



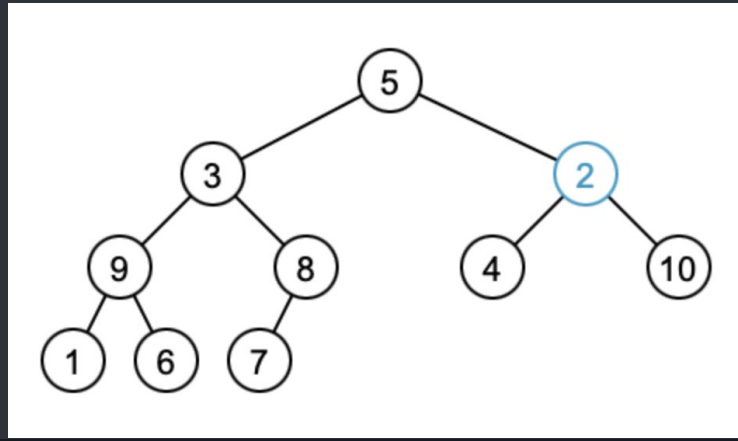
Запустим siftDown от первого элемента имеющего потомков, до корня

Пирамида.html

Эффективное построение пирамиды.css



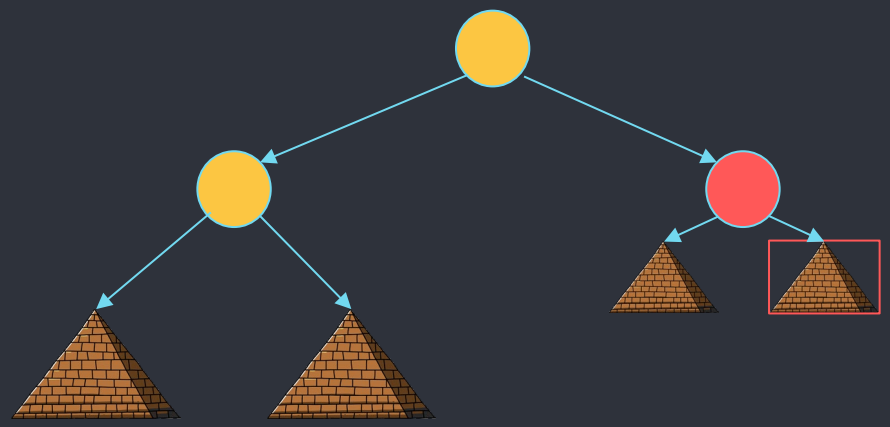
Запустим функцию siftDown для элементов от n/2 до 0



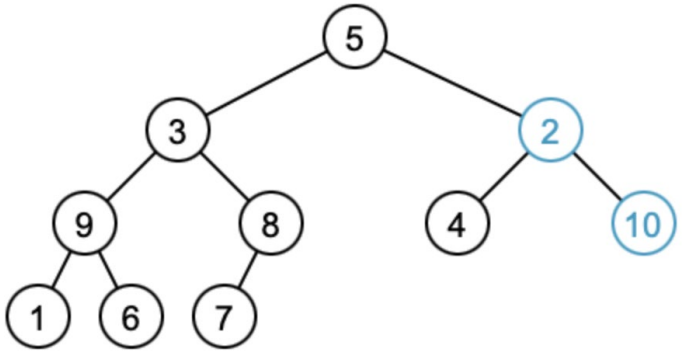
Запустим siftDown от первого элемента имеющего потомков, до корня

Пирамида.html

Эффективное построение пирамиды.css



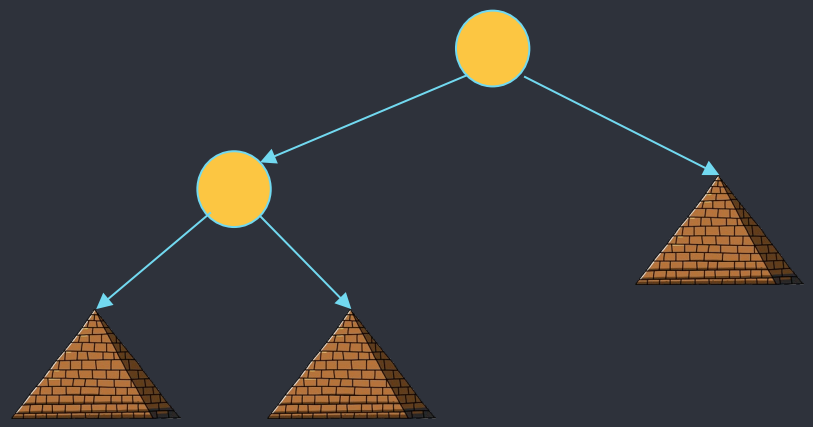
Запустим функцию siftDown для элементов от n/2 до 0



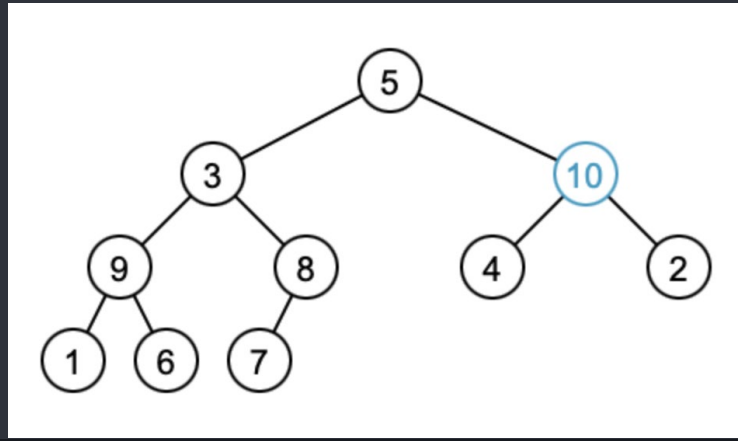
Запустим siftDown от первого элемента имеющего потомков, до корня

Пирамида.html

Эффективное построение пирамиды.css



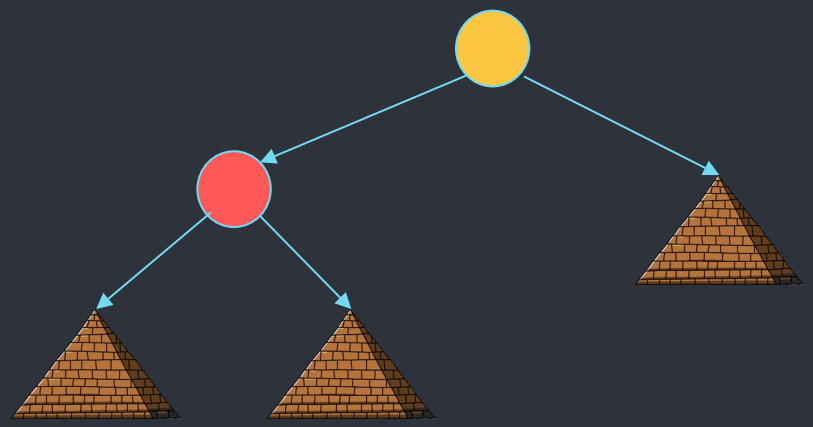
Запустим функцию siftDown для элементов от n/2 до 0



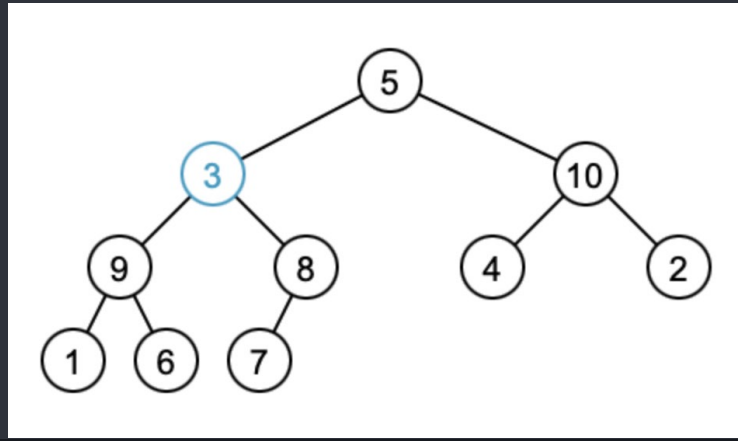
Запустим siftDown от первого элемента имеющего потомков, до корня

Пирамида.html

Эффективное построение пирамиды.css



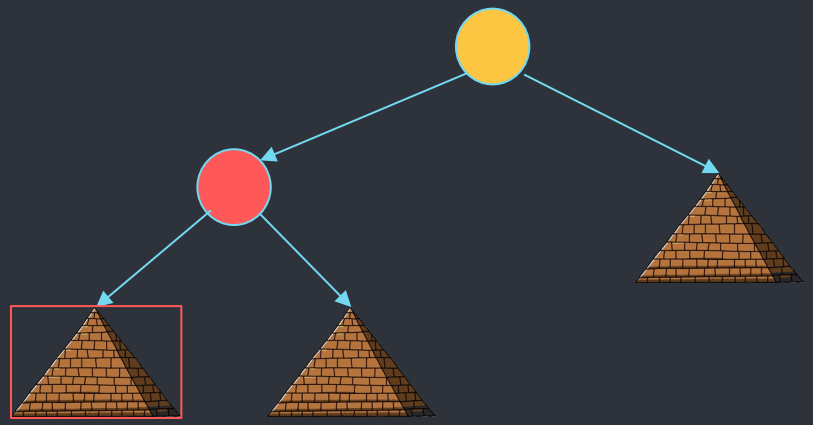
Запустим функцию siftDown для элементов от n/2 до 0



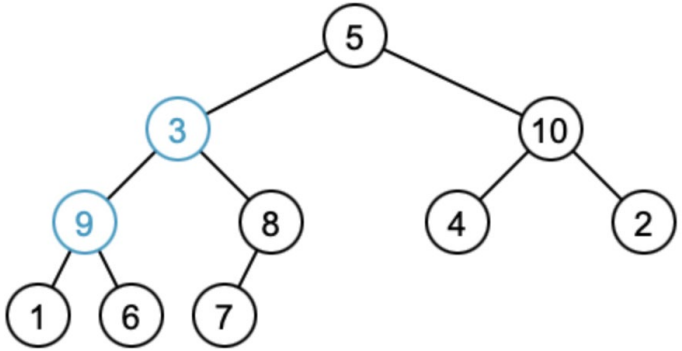
Запустим siftDown от первого элемента имеющего потомков, до корня

Пирамида.html

Эффективное построение пирамиды.css



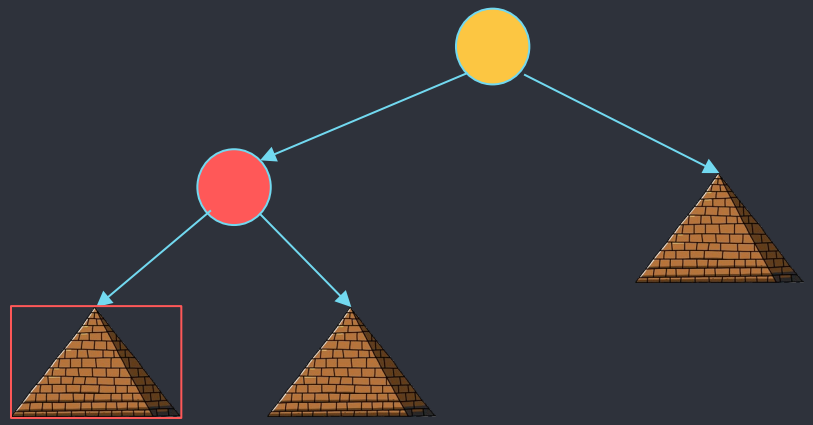
Запустим функцию siftDown для элементов от n/2 до 0



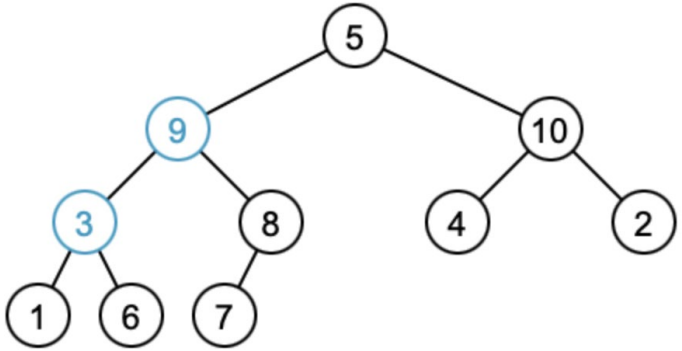
Запустим siftDown от первого элемента имеющего потомков, до корня

Пирамида.html

Эффективное построение пирамиды.css



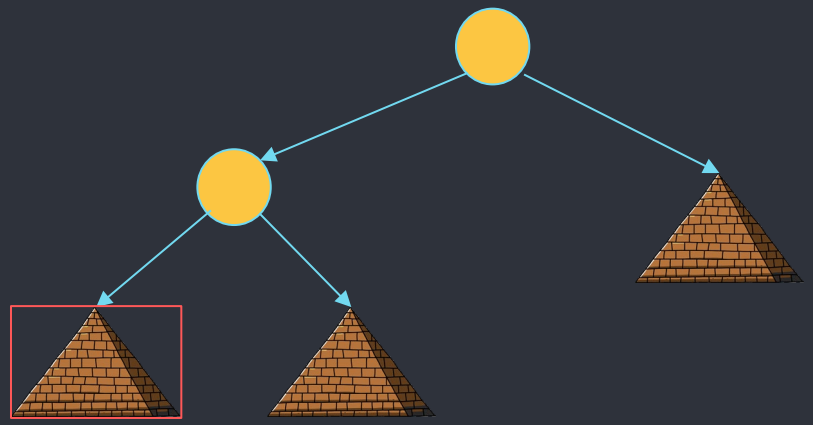
Запустим функцию siftDown для элементов от n/2 до 0



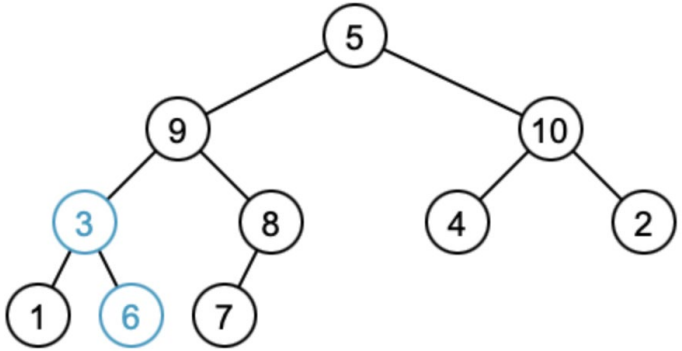
Запустим siftDown от первого элемента имеющего потомков, до корня

Пирамида.html

Эффективное построение пирамиды.css



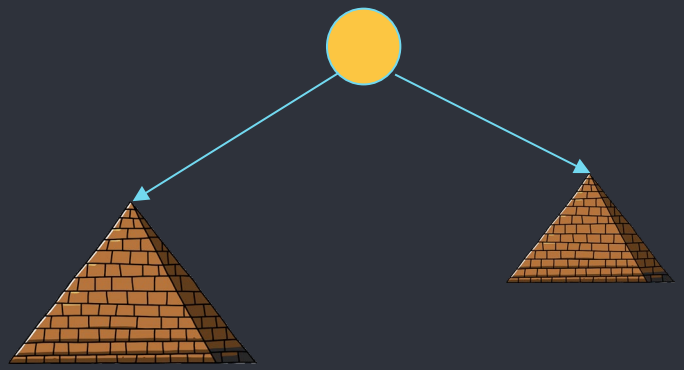
Запустим функцию siftDown для элементов от n/2 до 0



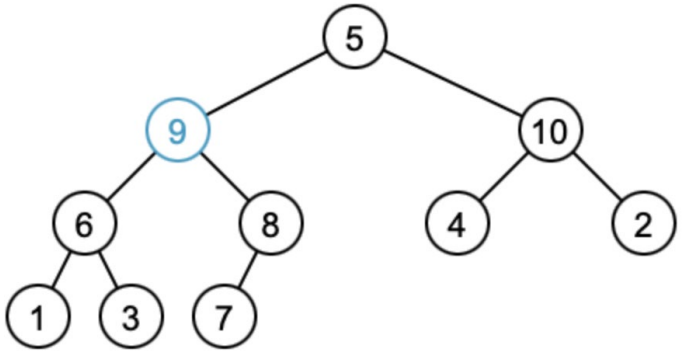
Запустим siftDown от первого элемента имеющего потомков, до корня

Пирамида.html

Эффективное построение пирамиды.css



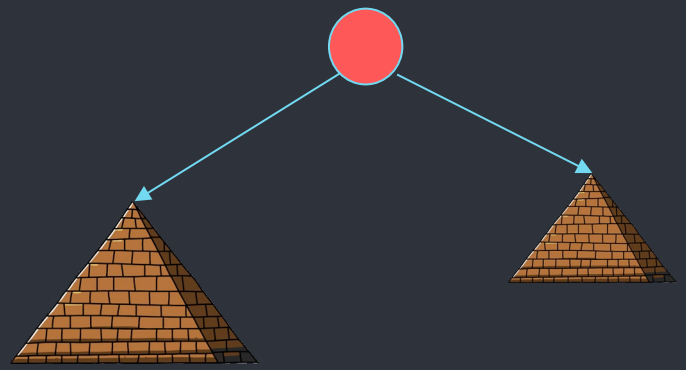
Запустим функцию siftDown для элементов от n/2 до 0



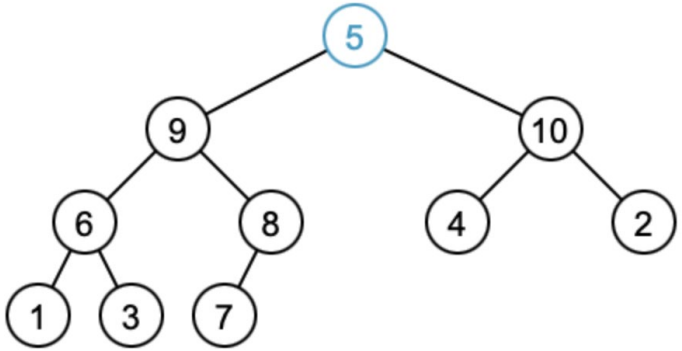
Запустим siftDown от первого элемента имеющего потомков, до корня

Пирамида.html

Эффективное построение пирамиды.css



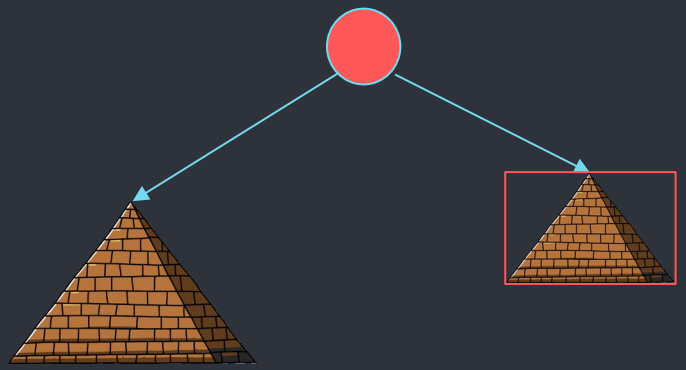
Запустим функцию siftDown для элементов от n/2 до 0



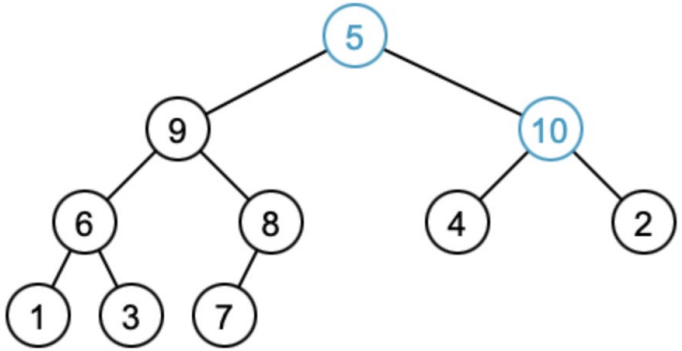
Запустим siftDown от первого элемента имеющего потомков, до корня

Пирамида.html

Эффективное построение пирамиды.css



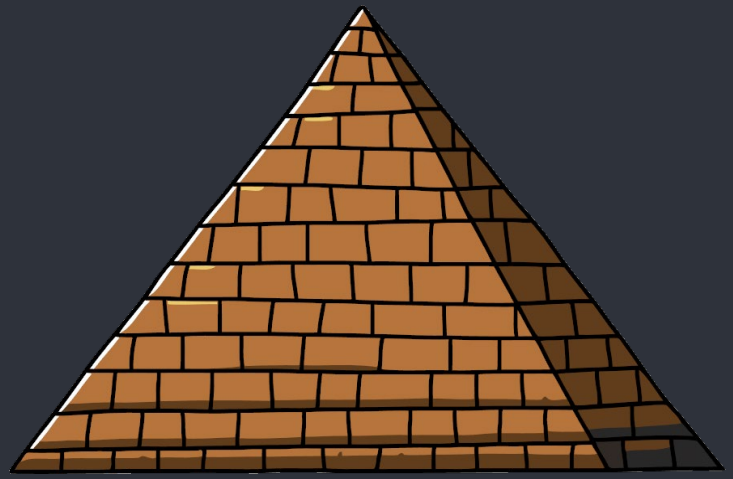
Запустим функцию siftDown для элементов от n/2 до 0



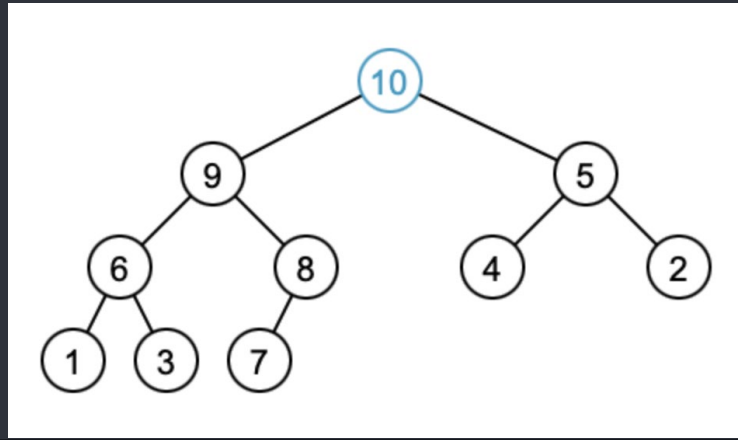
Запустим siftDown от первого элемента имеющего потомков, до корня

Пирамида.html

Эффективное построение пирамиды.css



Запустим функцию siftDown для элементов от n/2 до 0



Запустим siftDown от первого элемента имеющего потомков, до корня

Построение пирамиды за O(n) {

01 Запустим функцию siftDown для элементов от n/2 до 0

02 Операция siftDown работает за $O(\log_2 n)$

03 Запустили функцию, работающую за $O(\log_2 n)$ n/2 раз...

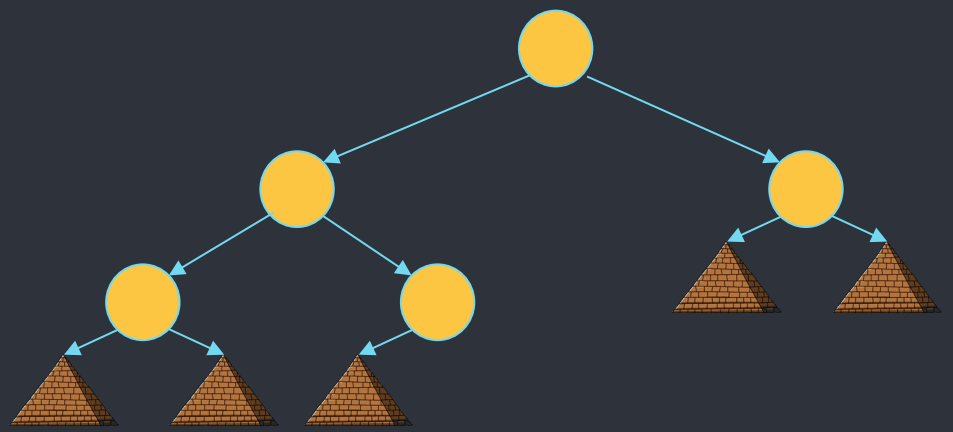
04 Разве асимптотическая оценка данного построение не равна $O(n*\log_2 n)$?

}

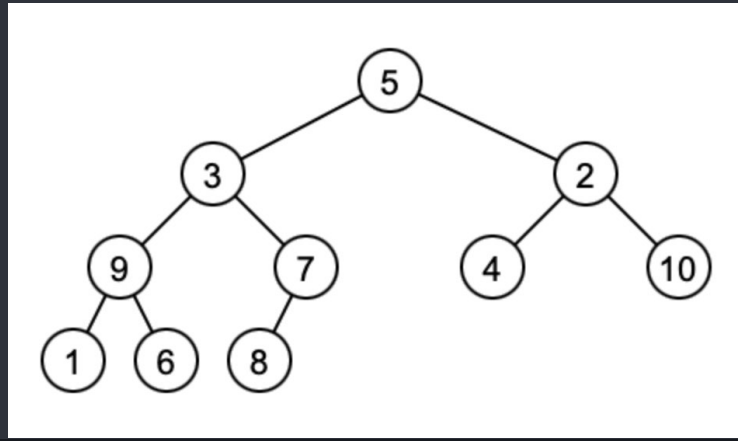
Давайте попробуем доказать, что на самом деле мы научились строить пирамиду за O(n)

Пирамида.html

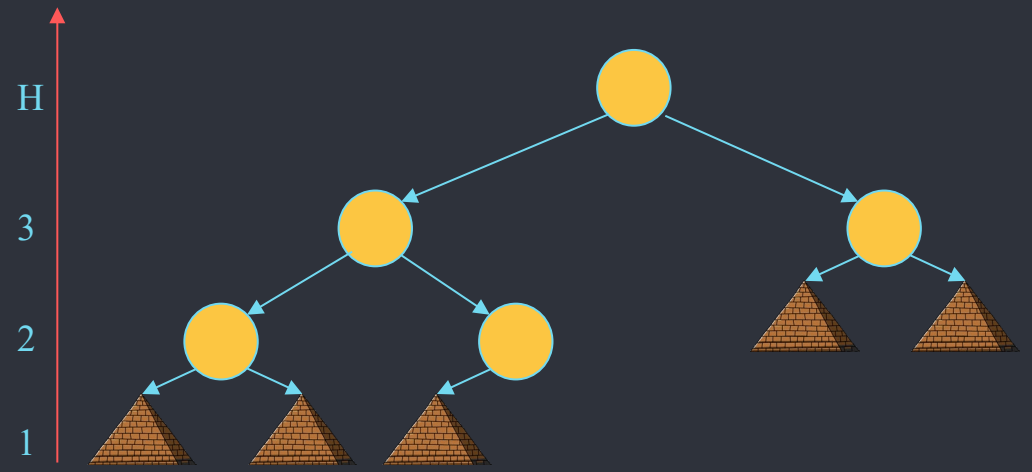
Эффективное построение пирамиды.css



Пусть H - высота завершенного бинарного дерева, на котором мы строим пирамиду => $H = \log_2 n$

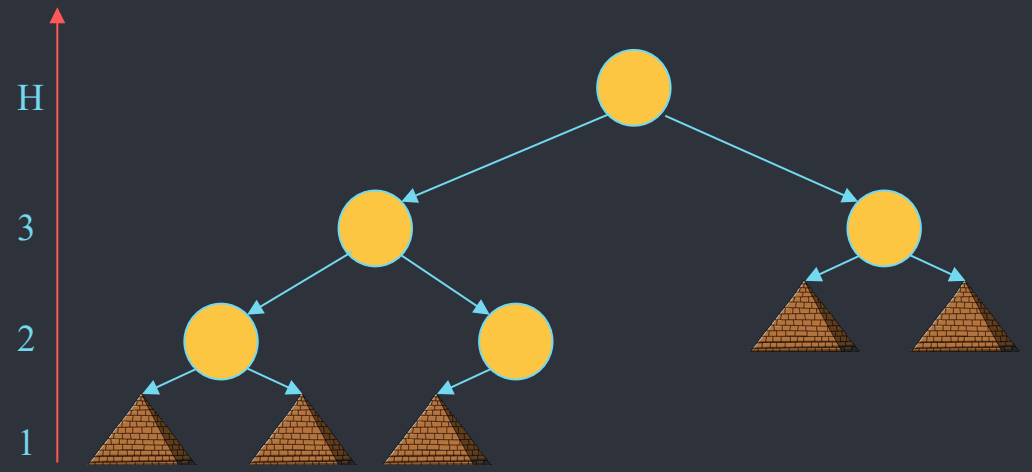


Запустим siftDown от первого элемента имеющего потомков, до корня



Обозначим уровни пирамиды (h) от основания пирамиды до корня, то есть от 1 до H

Тогда на каждом уровне вершин не более чем $n / 2^h$ (с округлением в большую сторону)



От каждой “не листовой” вершины мы запустили siftDown, но для всех вершин спуск возможен только на количество уровне не привышающих h

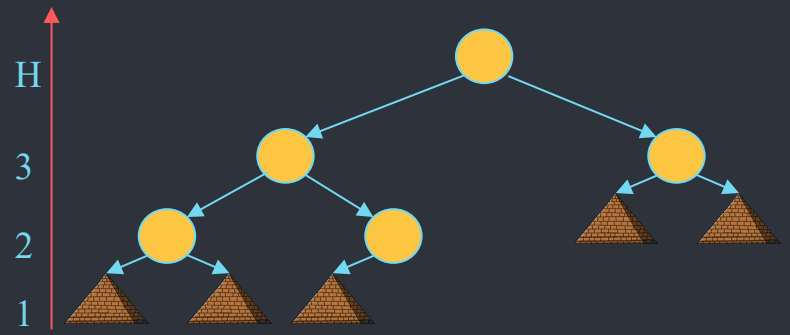
Тогда общее количество операций = сумме максимальных длин спусков от каждой из вершин

$$\sum_{h=1}^{\log_2 n} \frac{n}{2^h} \cdot 2 \cdot h$$

Количество вершин на уровне

Максимальное количество спусков при операци siftDown для вершине на уровне h

Операция siftDown сравнивает i элемент с двумя его детьми



Осталось только посчитать...

$$\sum_{h=1}^{\log_2 n} \frac{n}{2^h} \cdot 2 \cdot h = 2 \cdot n \cdot \sum_{h=1}^{\log_2 n} \frac{h}{2^h}$$

$$\sum_{h=1}^{\log_2 n} \frac{h}{2^h} \leq \sum_{h=1}^{\infty} \frac{h}{2^h}$$

$$\sum_{h=1}^{\infty} \frac{h}{2^h} = \frac{1}{2} + \frac{2}{4} + \frac{3}{8} + \frac{4}{16} + \frac{5}{32} + \ldots = 2$$

$$2 \cdot n \cdot \sum_{h=1}^{\log_2 n} \frac{h}{2^h} \leq 4 \cdot n = O(n)$$

ч. т. д.

Пирамида.html

Применение структуры данных: пирамида.css

1
2
3
4
5 < “Мы так подробно прошли структуру
6 данных “пирамида”. Давайте рассмотрим
7 ещё один способ, помимо пирамидальной
8 сортировки, применить наши знания и
9 умения на практике.” >
10
11 — Кто-то жадный до знаний в аудитории
12
13
14

Поздравляю, это 100-й слайд, мы на финишной прямой!

Что такое ‘Очередь с приоритетом’ {

01

Очередь с приоритетом — структура данных, в которой каждый элемент имеет приоритет.

02

Элементы извлекаются не в порядке добавления, а по их приоритету.

- Высокий приоритет = первый в очереди.
- Если приоритеты равны, используется порядок добавления.

03

Основные операции:

- Добавление (enqueue): добавление элемента с приоритетом.
- Извлечение (dequeue): удаление элемента с наивысшим приоритетом.

04

}

Звучит просто, да и с очередью уже работали. Почему бы не реализовать на массиве или на списке?

Реализация на списке; {

Наивная реализация:

- Используется упорядоченный список.
- Элементы вставляются так, чтобы поддерживать порядок по приоритету.

```
enqueue(element, priority):
    insert element in sorted order by priority

dequeue():
    return first element
```

}

Вставка: $O(n)$. Удаление: $O(1)$.

Реализация на массиве; {

Простая реализация:

- Элементы добавляются в конец массива.
- При извлечении выполняется поиск максимального/минимального элемента.

```
enqueue(element, priority):
    append element to array

dequeue():
    find and remove element with highest priority
```

}

Вставка: $O(1)$. Удаление: $O(n)$.

Реализация на пирамиде; {

Эффективная реализация:

- Используется пирамида, позволяющая извлекать максимальный/минимальный элемент за $O(\log_2 n)$.
- Элементы добавляются и извлекаются с соблюдением свойств пирамиды.

```
enqueue(element, priority):
    add element to heap
    restore heap property

dequeue():
    extract root of heap
    restore heap property
```

}

Вставка: $O(\log_2 n)$. Удаление: $O(\log_2 n)$.